

BAB I

SELECT & WHERE

1.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa memahami :

1. Tujuan pertama untuk dapat memahami cara menggunakan perintah `SELECT` untuk mendapatkan data dari tabel.
2. Tujuan kedua untuk dapat memahami cara menggunakan klausa `WHERE` untuk menyaring data berdasarkan kondisi tertentu.
3. Tujuan ketiga untuk dapat memahami penerapan operator aritmatika dalam SQL untuk melakukan perhitungan pada tabel.

1.2 Dasar Teori

Sebagai bahasa standar dalam manajemen basis data relasional, *Structured Query Language* (SQL) memfasilitasi pengguna untuk memproses data secara sistematis melalui aktivitas pengambilan, penyimpanan, pembaruan, hingga penghapusan informasi. Bahasa ini dikembangkan dengan mengutamakan kemudahan penggunaan sehingga memegang peranan fundamental dalam pengelolaan data di berbagai sistem basis data relasional (Santoso, 2021).

`SELECT` merupakan instruksi fundamental dalam SQL yang berfungsi untuk menyajikan data dari satu atau beberapa tabel di dalam basis data. Melalui perintah ini, pengguna memiliki fleksibilitas untuk menentukan kolom-kolom spesifik yang akan ditampilkan sesuai dengan keperluan analisis. Karena perannya dalam mengekstraksi informasi yang tersimpan, `SELECT` menjadi instrumen yang krusial dalam prosedur pencarian serta pengolahan data (Santoso, 2021).

Klausa `WHERE` diimplementasikan untuk menetapkan kriteria spesifik pada instruksi `SELECT`, sehingga data yang disajikan selaras dengan parameter yang dibutuhkan. Melalui penerapan `WHERE`, prosedur ekstraksi informasi menjadi lebih presisi karena sistem hanya akan menampilkan data yang lolos verifikasi syarat tertentu. Keberadaan klausa ini berperan penting dalam meningkatkan akurasi hasil *query* selama proses manipulasi basis data (Santoso, 2021).

Perintah `DESCRIBE` dalam SQL digunakan untuk melihat struktur sebuah tabel atau objek database, seperti nama kolom, tipe data, panjang data, serta status `NULL`. Perintah ini membantu pengguna memahami karakteristik tabel sebelum melakukan operasi pengolahan data dan sangat berguna dalam proses analisis serta pengelolaan basis data (Oracle, 2020).

Table aliasing dalam SQL yang disimbolkan dengan **AS** digunakan untuk memberikan nama sementara pada tabel dalam sebuah *query*. Teknik ini bertujuan meningkatkan keterbacaan dan efisiensi penulisan *query*, terutama pada *query* yang kompleks dan melibatkan banyak tabel, sehingga kode menjadi lebih ringkas dan mudah dipelihara (Catayoc, 2025).

Arithmetic Operators dalam SQL digunakan untuk melakukan operasi perhitungan numerik seperti penjumlahan, pengurangan, perkalian, dan pembagian. Operator ini dapat bersifat unary maupun binary, dengan ketentuan argumen bertipe numerik atau dapat dikonversi secara implisit, serta sering digunakan dalam perintah `SELECT` dan klausa `WHERE` untuk memproses data angka (Oracle, 2020).

Klausa `DISTINCT` digunakan setelah perintah `SELECT` untuk menampilkan data yang unik pada kolom tertentu. Setiap nilai yang sama hanya akan ditampilkan satu kali sehingga hasil *query* menjadi lebih ringkas, tanpa mengubah data asli di dalam tabel (Ciptayani, 2020).

Istilah `NULL` dalam SQL menunjukkan ketiadaan nilai pada suatu kolom dan berbeda dengan nol atau teks kosong. Untuk menangani kondisi ini, digunakan ekspresi `IS NULL` dan `IS NOT NULL` guna menampilkan data yang kolomnya kosong atau terisi, biasanya pada atribut yang bersifat opsional atau datanya belum tersedia (Ciptayani, 2020).

Concatenation Operator dalam SQL yang ditulis dengan simbol `||` digunakan untuk menggabungkan dua atau lebih string atau data bertipe `CLOB` menjadi satu teks. Operator ini sering digunakan pada perintah `SELECT` untuk memperjelas informasi yang ditampilkan, dengan hasil bertipe karakter sesuai argumen yang digunakan, serta mempertahankan spasi akhir string pada Oracle Database (Oracle, 2020).

Operator `BETWEEN` dalam SQL digunakan pada klausa `WHERE` untuk menyaring data berdasarkan rentang nilai tertentu. Penulisannya menggunakan kata `AND` sebagai pemisah batas awal dan akhir sehingga kondisi menjadi lebih jelas dan ringkas (Prayoga et al., 2023).

Operator `LIKE` dalam SQL digunakan untuk melakukan pencarian data berdasarkan pola tertentu, bukan kecocokan yang sama persis. Operator ini memanfaatkan karakter khusus seperti “%” dan “_” untuk mewakili bagian teks yang fleksibel dalam proses pencarian data (Fitri, 2020).

1.3 Hasil dan Pembahasan

Adapun beberapa percobaan *query* dari modul *section* yang sudah dipelajari yaitu sebagai berikut:

1. Query:

```
SELECT *  
FROM employees;
```

Hasil:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	BENEFITS
100	Steven	King	SKING	515.123.4567	00/11/1987	AD_PREST	24000	-	-	90	-
101	Neena	Kochhar	NKOCHHAR	515.123.4568	00/11/1988	AD_VP	17000	-	100	90	-
102	Lex	De Haan	LDEHAAN	515.123.4569	01/13/1993	AD_VP	17000	-	100	90	-
200	Jennifer	Whalen	JWHALEN	515.123.4564	00/11/1987	AD_ASST	4400	-	101	10	-
201	Shelley	Higgins	SHIGGINS	515.123.4560	00/01/1994	AC_MGR	12000	-	101	110	-
208	William	Gietz	WGIEZT	515.123.4561	00/01/1994	AC_ACCOUNT	8300	-	205	110	-
140	Ellen	Zlotkey	EZLOTKEY	011.44.1644.430258	01/02/2005	SA_MGR	10500	2	100	80	1600
174	Ellen	Abel	EABEL	011.44.1644.430267	00/11/1995	SA_REP	11000	3	140	80	1700
175	Jonathan	Taylor	JTAYLOR	011.44.1644.430255	03/04/1998	SA_REP	8000	2	140	80	1250
176	Kristianey	Grant	KGRANT	011.44.1644.430253	03/04/1998	SA_REP	7000	15	140	-	-
124	Kevin	Moursai	KMOURSAI	650.123.5234	11/08/1995	ST_MGR	5800	-	100	50	-
141	Tramma	Rajs	TRAJAS	650.121.8009	10/11/1995	ST_CLERK	3500	-	124	50	-
142	Gaillet	Dierkes	GDIRKES	650.121.2094	01/09/1997	ST_CLERK	3100	-	124	50	-
143	Randall	Mather	RMATHER	650.121.2074	01/01/1998	ST_CLERK	2900	-	124	50	-
144	Peter	Vargas	PVARGAS	650.121.2004	07/08/1998	ST_CLERK	2500	-	124	50	-
103	Alexander	Russell	ARUSSELL	590.423.4567	01/02/1995	IT_PROG	9000	-	102	60	-
104	Bruce	Ernst	BERNST	590.423.4568	00/01/1999	IT_PROG	6000	-	103	60	-
107	Diana	Lorentz	DLORENTZ	590.423.5567	02/01/1999	IT_PROG	4200	-	103	60	-
201	Michael	Hartstein	MHARTSTE	515.123.5555	02/11/1999	HR_MGR	13000	-	100	20	-
202	Pat	Fay	PFAY	650.123.6666	00/11/1997	HR_REP	6000	-	201	20	-

Gambar 1.1 Tampilan seluruh data pada tabel *employees*

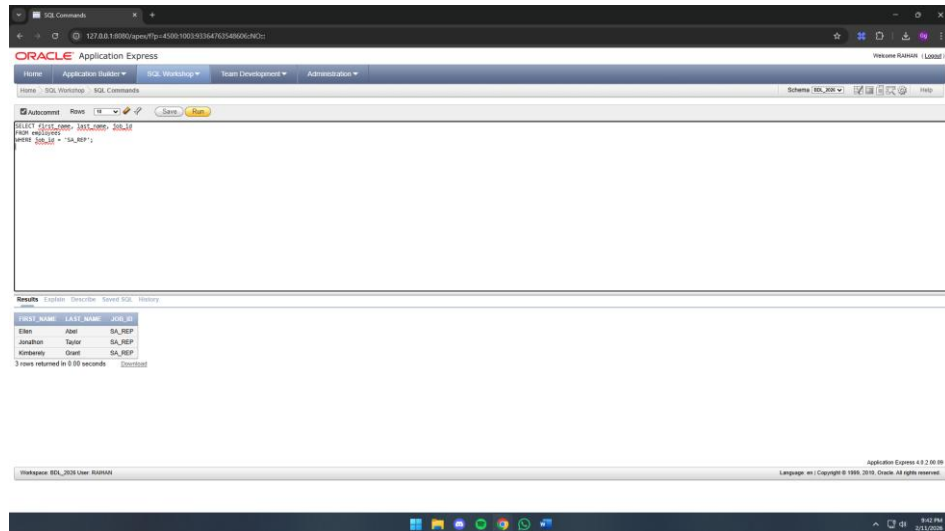
Penjelasan:

Query tersebut digunakan untuk menampilkan seluruh informasi yang tersimpan dalam tabel *employees*, di mana perintah *SELECT* berfungsi sebagai instruksi untuk mengambil data dari *database*. Penggunaan simbol asterisk (*) berperan sebagai penanda bahwa seluruh kolom akan ditampilkan tanpa pengecualian, sementara klausa *FROM employees* menunjukkan bahwa sumber data berasal dari tabel *employees*, sehingga hasil eksekusi perintah tersebut mencakup keseluruhan data yang tersedia di dalam tabel tersebut.

2. Query:

```
SELECT first_name, last_name, job_id
FROM employees
WHERE job_id = 'SA_REP';
```

Hasil:



Gambar 1.2 Tampilan hasil *query* `SELECT first_name, last_name, job_id` dengan kondisi `SA_REP`

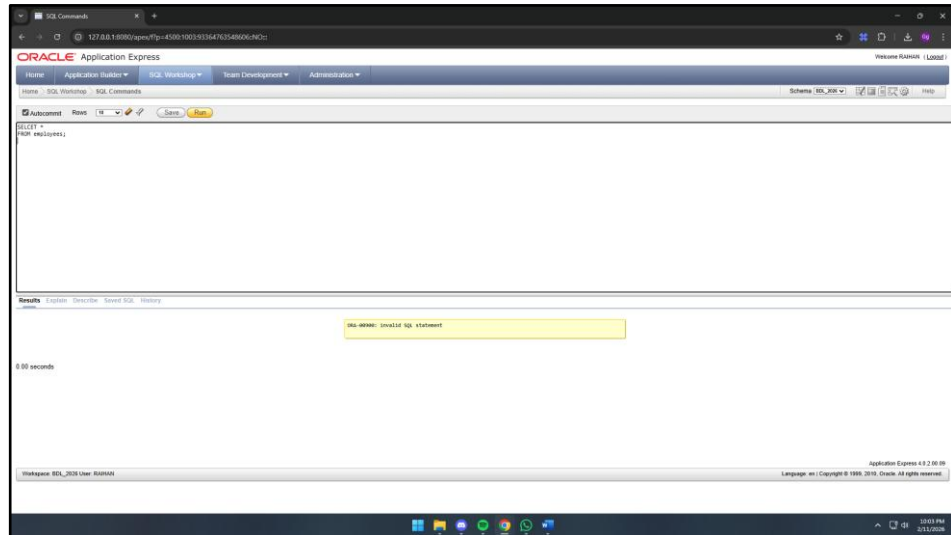
Penjelasan:

Query tersebut bertujuan untuk menampilkan data `first_name`, `last_name`, dan `job_id` yang berasal dari tabel `employees`. Proses penampilan data dilakukan secara selektif dengan menerapkan kondisi tertentu sehingga tidak seluruh rekaman karyawan ditampilkan. Data yang muncul terbatas pada karyawan dengan `job_id = 'SA_REP'`, sehingga hasil *query* tersebut secara khusus hanya menyajikan informasi bagi karyawan yang memiliki jabatan sebagai *Sales Representative*.

3. Query:

```
SELCT *  
FROM employees;
```

Hasil:



Gambar 1.3 Tampilan pesan *invalid SQL statement*

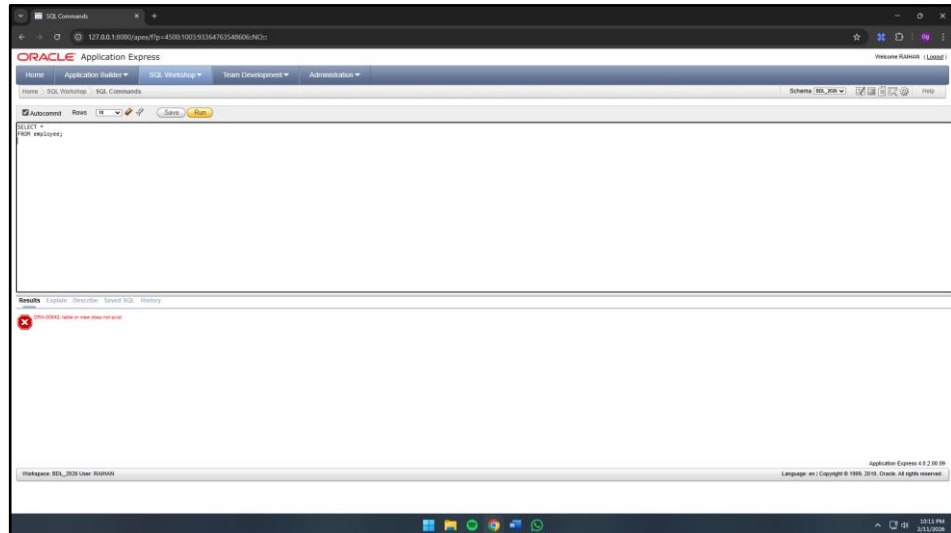
Penjelasan:

Query tersebut pada dasarnya dirancang untuk menampilkan keseluruhan data yang tersimpan di dalam tabel `employees`. Namun, akibat terjadinya kesalahan penulisan pada instruksi `SELECT`, sistem tidak dapat mengidentifikasi sintaks *Structured Query Language* (SQL) tersebut secara benar. Kekeliruan ini memicu munculnya pesan kesalahan berupa *invalid SQL statement*, sehingga proses eksekusi query mengalami kegagalan dan tidak menghasilkan *output* data apa pun.

4. Query:

```
SELECT *  
FROM employee;
```

Hasil:



Gambar 1.4 Tampilan pesan *error* pada *employee*

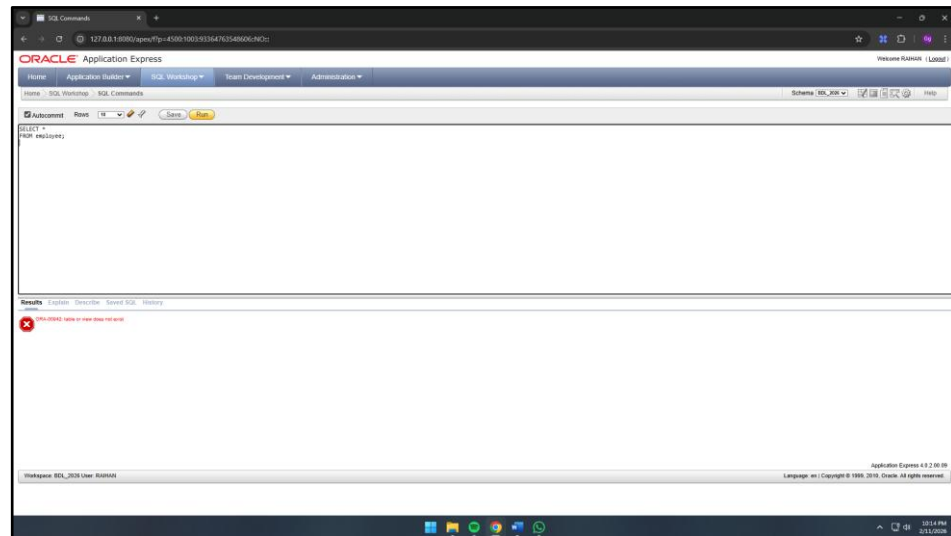
Penjelasan:

Query tersebut ditujukan untuk mengekstraksi keseluruhan data yang terdapat pada tabel *employees*. Akan tetapi, terdapat kekeliruan dalam penulisan identitas tabel yang menyebabkan sistem tidak mampu mengenali entitas tersebut di dalam *database*. Akibatnya, muncul pesan kesalahan berupa *table or view does not exist* yang menandakan bahwa proses eksekusi gagal dilakukan karena tabel tujuan tidak ditemukan.

5. Query:

```
SELECT name  
  
FROM employees;
```

Hasil:



Gambar 1.5 Tampilan pesan *invalid identifier*

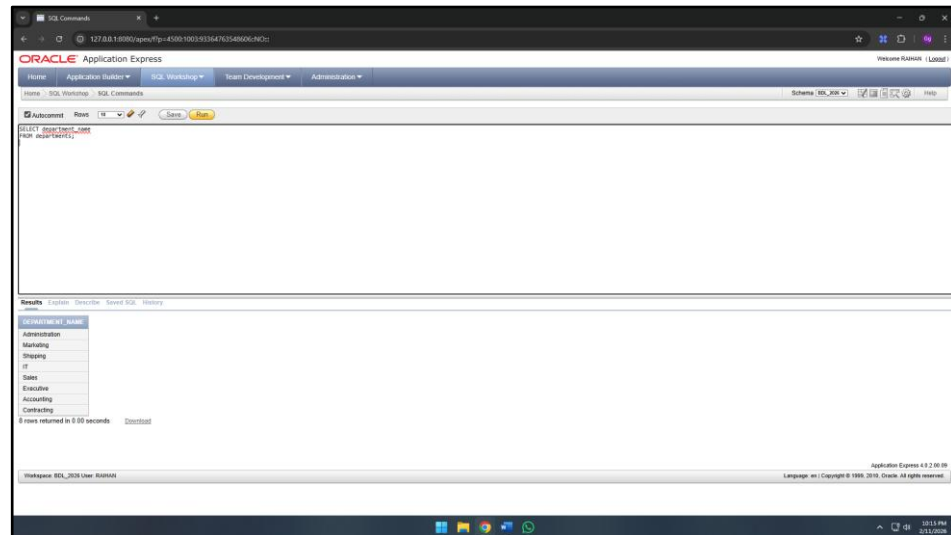
Penjelasan:

Query tersebut ditujukan untuk menampilkan data dari kolom `name` yang terdapat pada tabel `employees`. Namun, dikarenakan kolom tersebut secara faktual tidak tersedia di dalam skema tabel, instruksi *Structured Query Language* (SQL) tidak dapat diproses oleh sistem. Hal ini mengakibatkan terjadinya penolakan eksekusi *query* oleh *database*, sehingga tidak ada *output* data yang berhasil disajikan.

6. Query:

```
SELECT department_name  
  
FROM departments;
```

Hasil:



Gambar 1.6 Tampilan kolom `department_name` dari tabel `departments`

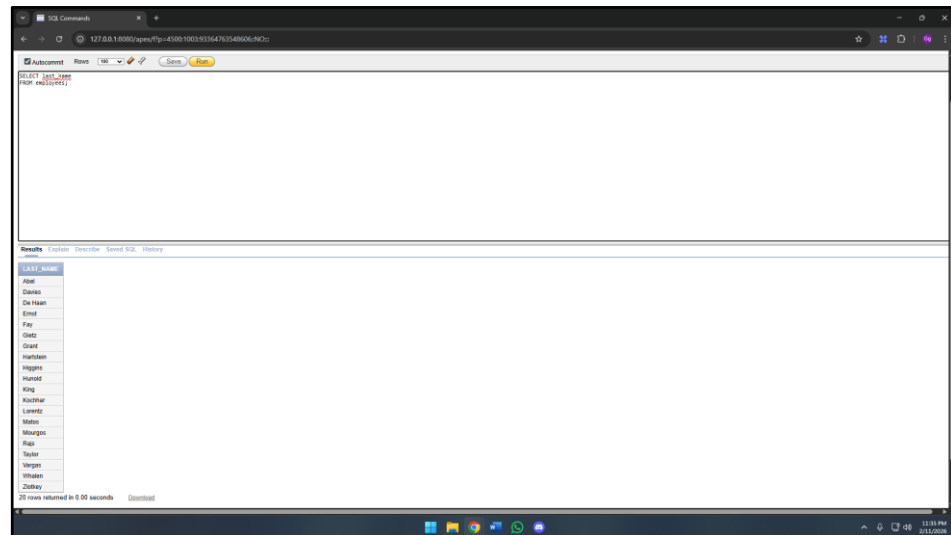
Penjelasan:

Query tersebut digunakan untuk mengekstraksi data `department_name` yang tersimpan di dalam tabel `departments`. Instruksi *Structured Query Language* (SQL) tersebut berhasil dieksekusi oleh sistem secara valid, sehingga menampilkan daftar nama departemen yang terdaftar secara komprehensif. Melalui implementasi perintah ini, seluruh informasi mengenai nama departemen yang tersedia di dalam *database* dapat disajikan sebagai *output* yang akurat.

7. Query:

```
SELECT last_name  
FROM employees;
```

Hasil:



LAST_NAME
Abad
Davies
De Haan
Ernst
Fay
Grant
Hartstein
Higgins
Hurold
King
Kochhar
Kumar
Males
Merges
Neel
Teller
Verger
Whalen
Zlotkey

Gambar 1.7 Tampilan kolom `last_name` dari tabel `employees`

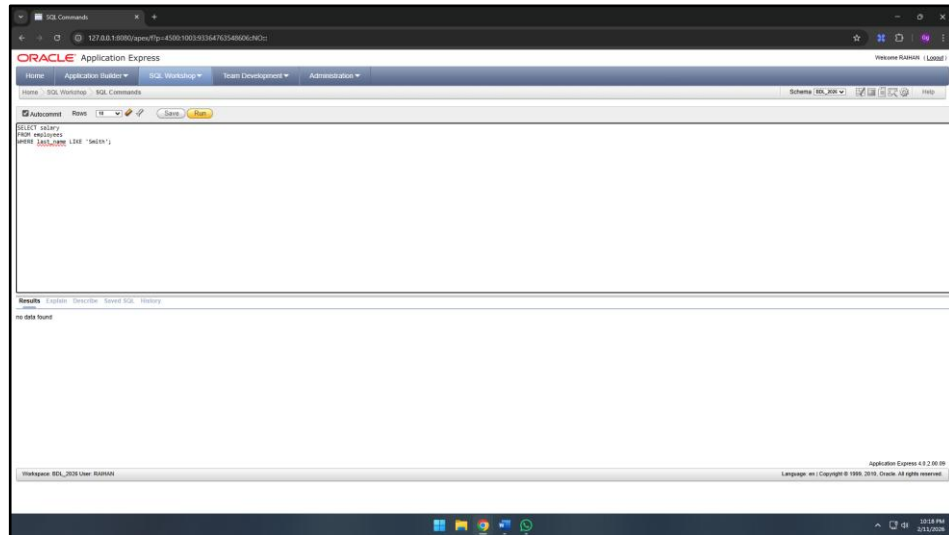
Penjelasan:

Query tersebut ditujukan untuk menampilkan informasi pada kolom `last_name` yang terdapat di dalam tabel `employees`. Instruksi *Structured Query Language* (SQL) tersebut berhasil dieksekusi oleh sistem sehingga menyajikan daftar nama belakang seluruh karyawan secara mendetail. Hasil dari *query* ini mencakup keseluruhan baris data yang tersedia, di mana jumlah *output* yang dihasilkan selaras dengan total rekaman karyawan yang tersimpan di dalam *database*.

8. Query:

```
SELECT salary  
FROM employees  
WHERE last_name LIKE 'Smith';
```

Hasil:



Gambar 1.8 Tampilan hasil *query* salary dengan kondisi last_name
LIKE 'Smith'

Penjelasan:

Query tersebut digunakan untuk menampilkan informasi salary dari tabel employees dengan menerapkan filter spesifik melalui kondisi last_name LIKE 'Smith'. Meskipun instruksi *Structured Query Language* (SQL) tersebut berhasil dieksekusi secara teknis oleh sistem, namun tidak ada data yang dihasilkan pada *output*. Hal ini terjadi dikarenakan tidak terdapat rekaman karyawan di dalam *database* yang memenuhi pola pencarian tersebut, sehingga kondisi yang didefinisikan tidak cocok dengan satu pun baris data yang tersedia.

9. Query:

```
SELECT *  
FROM countries;
```

Hasil:

COUNTRY_ID	COUNTRY	REGION_ID
CA	Canada	2
DE	Germany	1
UK	United Kingdom	1
US	United States of America	2

Gambar 1.9 Tampilan seluruh data tabel `countries`

Penjelasan:

Query tersebut bertujuan untuk mengekstraksi keseluruhan data yang tersimpan di dalam tabel `countries`. Instruksi *Structured Query Language* (SQL) tersebut berhasil dieksekusi dengan valid, sehingga menyajikan informasi komprehensif mengenai setiap negara sesuai dengan rekaman yang ada. Melalui implementasi perintah ini, pengguna dapat meninjau seluruh kolom serta baris yang tersedia secara lengkap pada tabel `countries` melalui *database*.

10. Query:

```
SELECT country_id, country_name, region_id  
FROM countries;
```

Hasil:

The screenshot shows the Oracle Application Express interface. The SQL Commands window contains the query: `SELECT country_id, country_name, region_id FROM countries;`. The Results window displays the following data:

COUNTRY_ID	COUNTRY_NAME	REGION_ID
CA	Canada	2
DE	Germany	1
UK	United Kingdom	1
US	United States of America	2

4 rows returned in 0.00 seconds

Gambar 1.10 Tampilan hasil *query* `SELECT country_id, country name, dan region_id` dari tabel `countries`

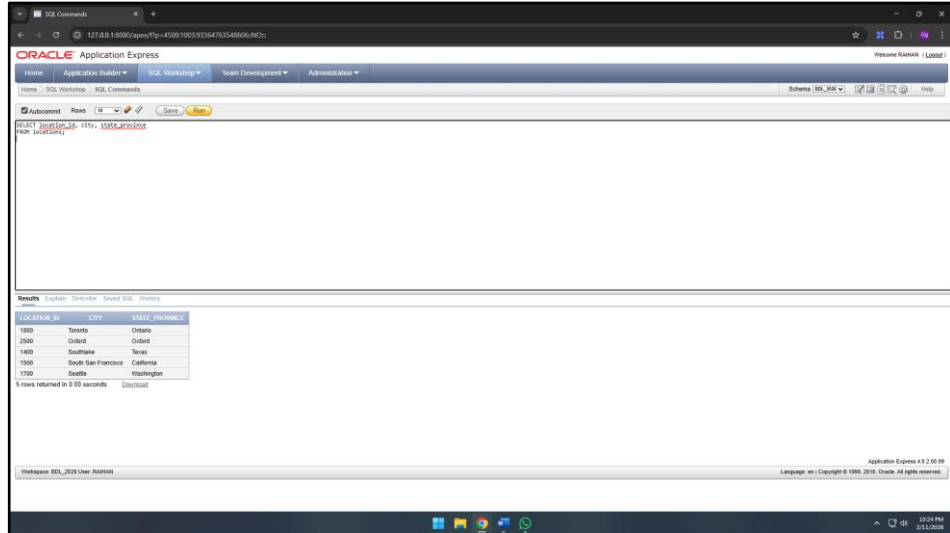
Penjelasan:

Query tersebut digunakan untuk mengekstraksi kolom `country_id`, `country_name`, dan `region_id` yang terdapat pada tabel `countries`. Perintah ini memungkinkan pengguna untuk meninjau daftar seluruh negara beserta identitas *region* atau wilayah terkait yang tersimpan di dalam tabel tersebut. Melalui implementasi instruksi ini, hubungan antara setiap negara dan unit wilayah yang tercatat di dalam *database* dapat teridentifikasi secara jelas.

11. Query:

```
SELECT location_id, city, state_province  
FROM locations;
```

Hasil:



LOCATION_ID	CITY	STATE_PROVINCE
1800	Toronto	Ontario
2500	Oxford	Oxford
1400	Southampton	Texas
1900	South San Francisco	California
1700	Seattle	Washington

Gambar 1.11 Tampilan hasil *query* `SELECT location_id, city, dan state_province` dari tabel `locations`

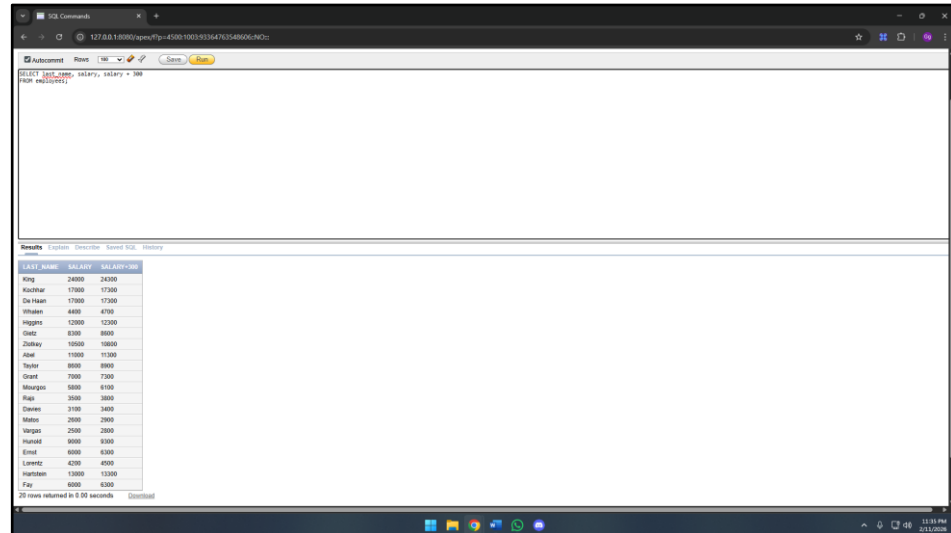
Penjelasan:

Query ini digunakan untuk menampilkan kolom `location_id`, `city`, dan `state_province` dari tabel `locations`. Dengan perintah ini, pengguna dapat melihat daftar lokasi beserta kota dan *state* atau provinsi yang tercatat di tabel. Cara ini memudahkan untuk memahami distribusi geografis setiap lokasi yang tersimpan dalam basis data.

12. Query:

```
SELECT last_name, salary, salary + 300  
FROM employees;
```

Hasil:



LAST_NAME	SALARY	SALARY+300
King	24000	24300
Kochhar	17000	17300
De Haan	17000	17300
Whalen	4800	5100
Higgins	12000	12300
Gietz	8300	8600
Zlotkey	10500	10800
Abel	11000	11300
Taylor	8800	9100
Grant	7800	8100
Mourgos	9800	10100
Raph	3600	3900
Davies	3100	3400
Mates	2800	3100
Verges	2900	3200
Hunold	9000	9300
Ernst	8000	8300
Lavigne	4200	4500
Hartstein	13000	13300
Fay	8500	8800

Gambar 1.12 Tampilan hasil *query* `SELECT last_name, salary, dan salary + 300`

Penjelasan:

Query ini digunakan untuk menampilkan kolom `last_name`, `salary`, dan hasil penjumlahan `salary + 300` dari tabel *employees*. Dengan perintah ini, pengguna dapat melihat nama belakang karyawan beserta gaji mereka, sekaligus menampilkan simulasi kenaikan gaji sebesar 300. Cara ini memudahkan analisis dan perbandingan gaji karyawan secara cepat.

13. Query:

```
SELECT last_name, salary, 12*salary +100  
FROM employees;
```

Hasil:

last_name	salary	12*salary +100
King	24000	288100
Kochhar	17000	204100
De Haan	17000	204100
Whalen	4800	528100
Higgins	12000	144100
Olsen	8300	987100
Zlotkey	10500	126100
Abel	11000	132100
Taylor	8500	103100
Grant	7800	84100
Moursun	5800	697100
Rajs	3500	42100
Green	3100	372100
Matsuo	2600	313100
Vargas	2500	30100
Howard	8500	103100
Ernst	8000	92100
Lorentz	4200	509100
Hartstein	13000	156100
Fay	8000	92100

Gambar 1.13 Tampilan hasil *query* SELECT last_name, salary, dan 12*salary +100

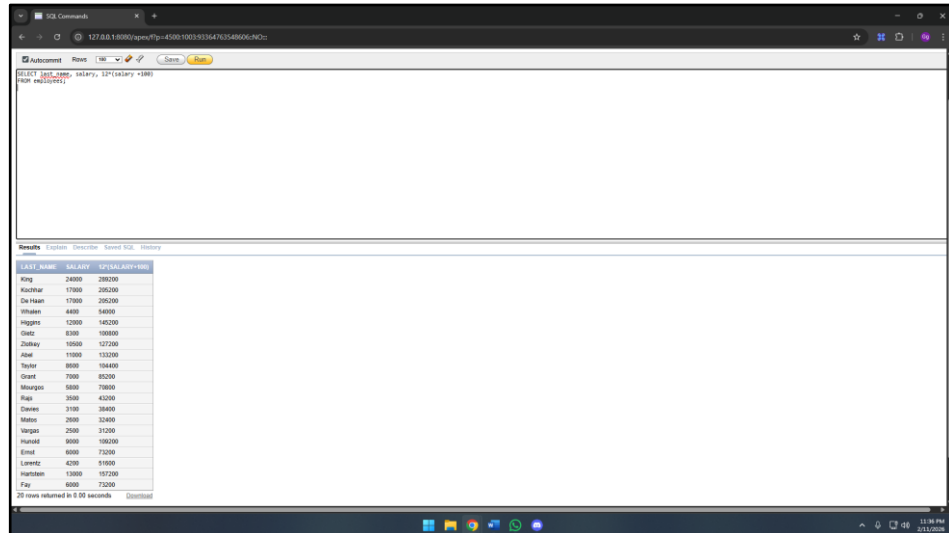
Penjelasan:

Query ini digunakan untuk menampilkan kolom last_name, salary, serta hasil perhitungan $12 * salary + 100$ dari tabel employees. Dengan perintah ini, pengguna dapat melihat nama belakang karyawan beserta gaji mereka, sekaligus menampilkan simulasi total gaji tahunan ditambah tambahan 100. Cara ini memudahkan analisis perhitungan gaji karyawan secara keseluruhan.

14. Query:

```
SELECT last_name, salary, 12*(salary +100)
FROM employees;
```

Hasil:



LAST_NAME	SALARY	12*(SALARY+100)
King	24000	288000
Kochhar	17000	204000
De Haan	17000	204000
Vihalen	4800	54000
Higgins	12000	144000
Gietz	6300	75600
Zlotkey	10500	127200
Abel	11000	133200
Taylor	8000	96000
Grant	7000	84000
Murphy	5800	69600
Raj	2600	31200
Green	3100	37200
Mates	2600	31200
Verges	2900	34800
Hunold	9000	108000
Ernst	8000	96000
Lavigne	4200	50400
Hartstein	13000	156000
Fay	6000	72000

Gambar 1.14 Tampilan hasil *query* `SELECT last_name, salary, dan 12*(salary +100)`

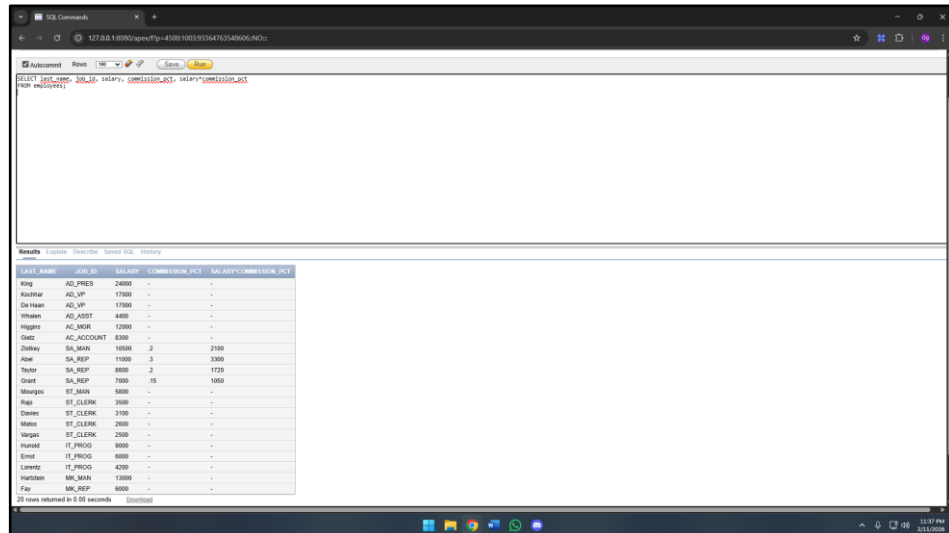
Penjelasan:

Query ini digunakan untuk menampilkan kolom `last_name`, `salary`, serta hasil perhitungan `12 * (salary + 100)` dari tabel `employees`. Dengan perintah ini, pengguna dapat melihat nama belakang karyawan beserta gaji mereka, sekaligus menampilkan simulasi total gaji tahunan setelah menambahkan bonus 100 pada setiap gaji bulanan. Cara ini mempermudah analisis proyeksi gaji tahunan karyawan.

15. Query:

```
SELECT last_name, job_id, salary, commission_pct,  
salary*commission_pct  
FROM employees;
```

Hasil:



LAST_NAME	JOB_ID	SALARY	COMMISSION_PCT	SALARY*COMMISSION_PCT
King	AD_PRES	24000	-	-
Kozmin	AD_VP	17000	-	-
De Haan	AD_VP	17000	-	-
Venkat	AD_VP	17000	-	-
Higgins	AC_MGR	12000	-	-
Gietz	AC_ACCOUNT	8300	-	-
Cheney	SA_MAN	10000	2	2000
Abel	SA_REP	11000	3	3300
Taylor	SA_REP	8900	2	1780
Grant	SA_REP	7800	15	1170
Mourges	ST_MAN	5800	-	-
Rais	ST_CLERK	3500	-	-
Deves	ST_CLERK	3100	-	-
Males	ST_CLERK	2600	-	-
Verges	ST_CLERK	2500	-	-
Hartel	IT_PROG	8600	-	-
Ernst	IT_PROG	6000	-	-
Lorentz	IT_PROG	4200	-	-
Hartstein	HR_MAN	13000	-	-
Fay	HR_REP	6000	-	-

Gambar 1.15 Tampilan query SELECT last_name, job_id, salary salary, commission_pct, dan salary*commission_pct

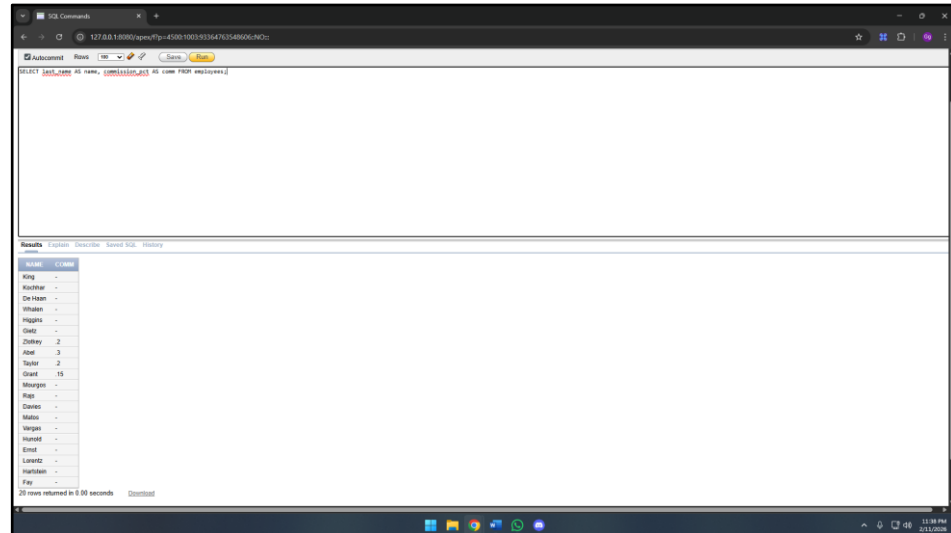
Penjelasan:

Query ini digunakan untuk menampilkan kolom last_name, job_id, salary, commission_pct, serta hasil perkalian salary * commission_pct dari tabel employees. Perintah ini memungkinkan pengguna melihat nama belakang karyawan, jabatan mereka, gaji, persentase komisi, dan total komisi yang diperoleh dari gaji. Dengan begitu, analisis kinerja dan pendapatan tambahan karyawan dapat dilakukan secara langsung dan jelas.

16. Query:

```
SELECT last_name AS name, commission_pct AS comm  
FROM employees;
```

Hasil:



NAME	COMM
King	-
Kochhar	-
De Haan	-
Whalen	-
Higgins	-
Giet	-
Zlotney	2
Abel	3
Taylor	2
Grant	15
Morgan	-
Rais	-
Coates	-
Maiden	-
Verges	-
Hunold	-
Ernst	-
Lavigne	-
Hartstein	-
Fay	-

Gambar 1.16 Tampilan kolom `last_name` sebagai `name` dan `commission_pct` sebagai `comm`

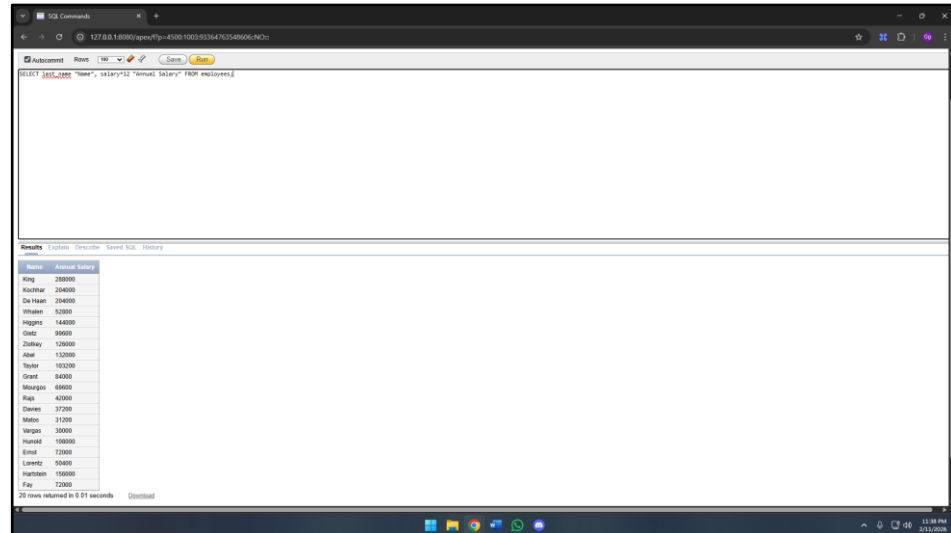
Penjelasan:

Query ini digunakan untuk menampilkan kolom `last_name` dan `commission_pct` dari tabel `employees`, dengan masing-masing kolom diberi alias `name` dan `comm`. Perintah ini memungkinkan pengguna melihat nama belakang karyawan beserta persentase komisi mereka dengan penamaan kolom yang lebih mudah dipahami. Cara ini memudahkan pembacaan hasil *query* saat menampilkan data dengan alias yang jelas.

17. Query:

```
SELECT last_name "Name", salary*12 "Annual Salary"
FROM employees;
```

Hasil:



Name	Annual Salary
King	280000
Kochhar	254000
De Haan	244000
Whalen	52000
Higgins	144000
Giet	99000
Zlotkey	120000
Abel	132000
Taylor	103000
Grant	84000
Mourgos	69000
Rap	42000
Dierkes	37200
Mates	31200
Verges	30000
Hunold	108000
Ernst	72000
Lynce	24000
Hartstein	100000
Fay	72000

Gambar 1.17 Tampilan kolom last_name sebagai Name dan salary*12 sebagai Annual Salary

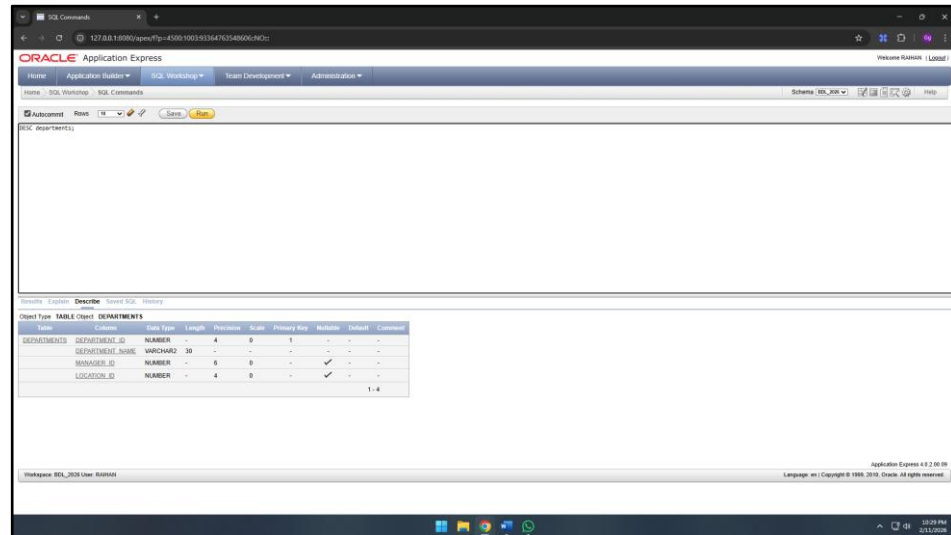
Penjelasan:

Query ini digunakan untuk menampilkan kolom last_name dan hasil perhitungan salary * 12 dari tabel employees, dengan alias Name untuk nama belakang karyawan dan Annual Salary untuk gaji tahunan. Perintah ini memungkinkan pengguna melihat nama karyawan sekaligus estimasi total gaji mereka selama setahun. Dengan cara ini, informasi mengenai pendapatan tahunan karyawan dapat dipahami dengan lebih jelas dan terstruktur.

18. Query:

```
DESC departments;
```

Hasil:



The screenshot shows the Oracle SQL Developer interface. The 'Tools' menu is open, and the 'SQL Worksheet' option is selected. The main window displays the 'DESC departments;' query. Below the query, the 'Structure' tab is active, showing the table structure of 'DEPARTMENTS'. The table has four columns: DEPARTMENT_ID, DEPARTMENT_NAME, MANAGER_ID, and LOCATION_ID. The data types and constraints are as follows:

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPARTMENTS	DEPARTMENT_ID	NUMBER	-	4	0	1	-	-	-
DEPARTMENTS	DEPARTMENT_NAME	VARCHAR2	30	-	-	-	-	-	-
DEPARTMENTS	MANAGER_ID	NUMBER	-	6	0	-	-	-	-
DEPARTMENTS	LOCATION_ID	NUMBER	-	4	0	-	-	-	-

Gambar 1.18 Tampilan deskripsi tabel departments

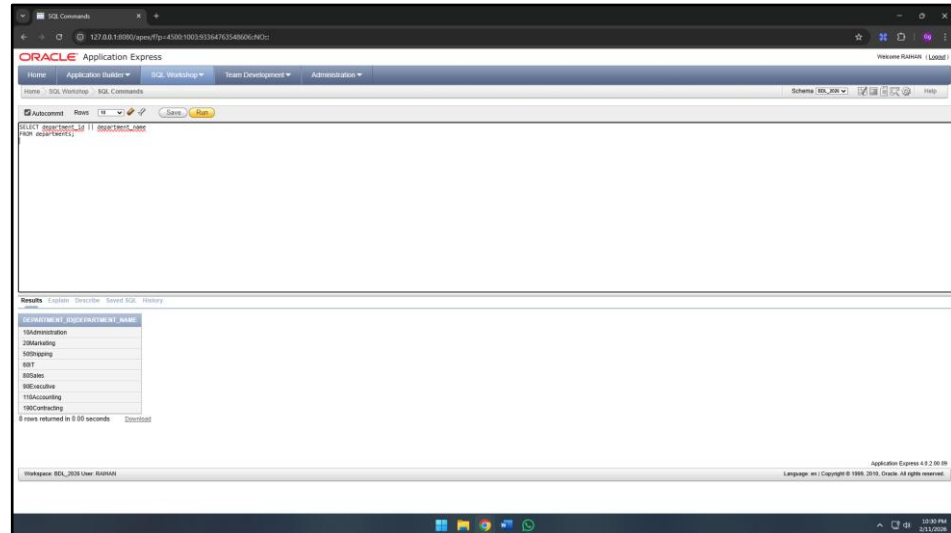
Penjelasan:

Query tersebut digunakan untuk menampilkan struktur tabel departments yang berisi informasi nama kolom, tipe data, dan atribut setiap kolom. Cara ini membantu memahami susunan dan karakteristik setiap kolom dalam tabel sebelum melakukan operasi data.

19. Query:

```
SELECT department_id || department_name  
FROM departments;
```

Hasil:



DEPARTMENT_ID	DEPARTMENT_NAME
10	Administration
20	Marketing
30	Shipping
40	IT
50	Sales
60	Executive
70	Accounting
80	Contracting

Gambar 1.19 Tampilan kolom departments_id dan departments_name yang digabungkan

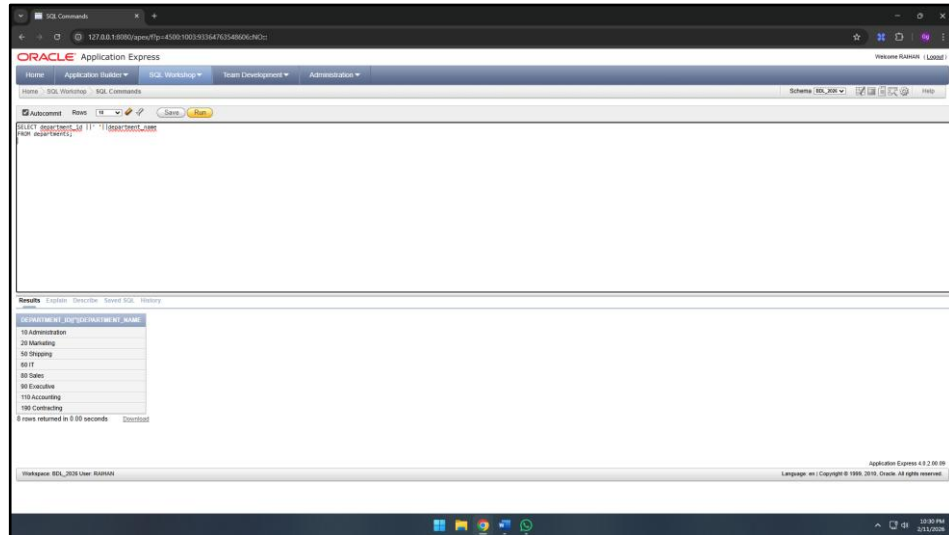
Penjelasan:

Query tersebut digunakan untuk menampilkan data departments yang terdiri dari department_id dan department_name. Query tersebut menggunakan simbol || untuk menggabungkan beberapa nilai atau kolom menjadi satu hasil tampilan.

20. Query:

```
SELECT department_id || ' ' || department_name  
FROM departments;
```

Hasil:



Gambar 1.20 Tampilan kolom departments_id dan departments_name yang digabungkan dengan pemisah spasi

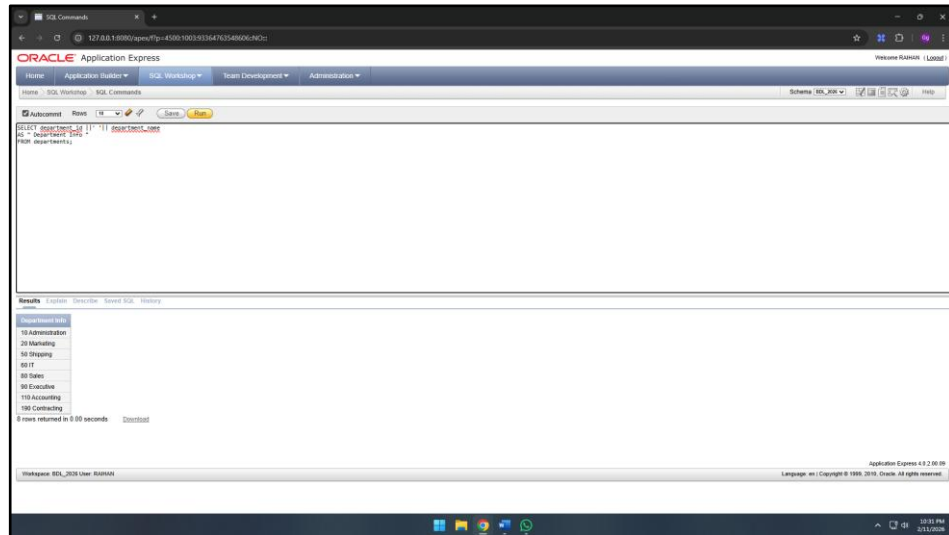
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom departments yang terdiri dari department_id dan department_name. Query tersebut menggabungkan kedua kolom tersebut menggunakan operator || ' ' || sehingga hasil tampilan berada dalam satu kolom dengan pemisah spasi. Dengan cara ini, pengguna dapat melihat secara bersamaan dalam satu tampilan, sehingga informasi lebih ringkas dan mudah dibaca.

21. Query:

```
SELECT department_id || ' ' || department_name  
AS " Department Info "  
FROM departments;
```

Hasil:



Gambar 1. 21 Tampilan kolom departments_id dan departments_name yang digabungkan sebagai kolom “Department Info”

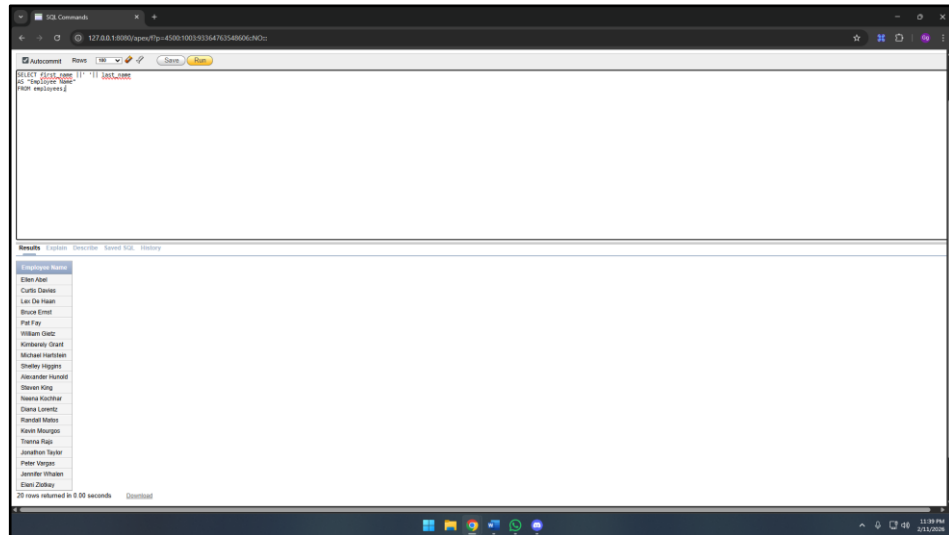
Penjelasan:

Query tersebut digunakan untuk menampilkan data department_id dan department_name dari tabel departments, dengan menambahkan alias “Department Info” pada kolom hasil. Sehingga informasi yang ditampilkan lebih jelas dan mudah dipahami.

22. Query:

```
SELECT first_name || ' ' || last_name  
AS "Employee Name"  
FROM employees;
```

Hasil:



Employee Name
Elton Abel
Curtis Davies
Lee Du Haan
Bruce Ernst
Pat Fay
William Gietz
Kimbberly Grant
Michael Hartman
Shelley Higgins
Alexander Hunold
Steven King
Neena Kochhar
Diana Lorentz
Randall Mates
Karen Morgan
Thomas Rupp
Jonathan Taylor
Peter Vanotti
Jennifer Whalen
Elvira Zlotkey

Gambar 1.22 Tampilan kolom `first_name` dan `last_name` yang digabungkan sebagai kolom “Employee Name”

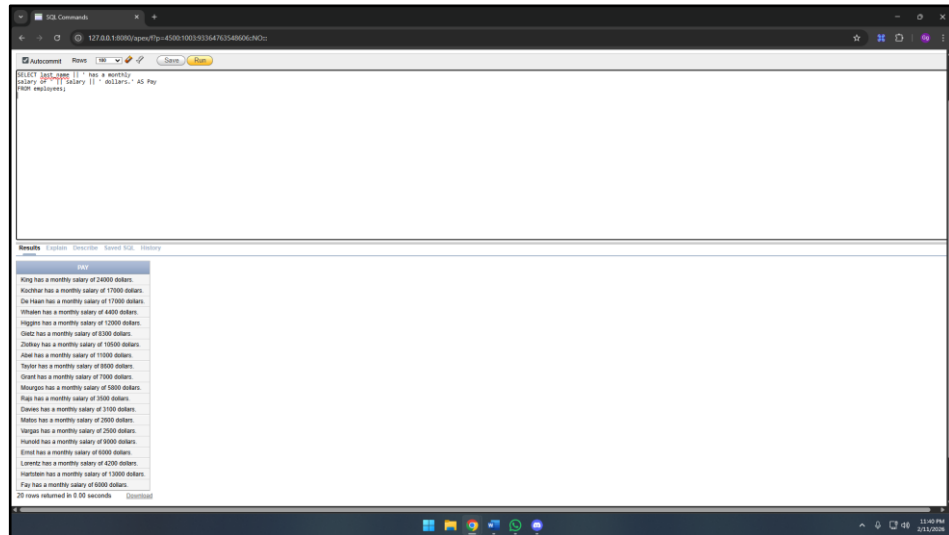
Penjelasan:

Query ini digunakan untuk menampilkan gabungan kolom `first_name` dan `last_name` dari tabel `employees`, dengan alias “Employee Name” pada kolom hasil. Sehingga nama karyawan ditampilkan lengkap dalam satu kolom.

23. Query:

```
SELECT last_name || ' has a monthly  
salary of ' || salary || ' dollars.' AS Pay  
FROM employees;
```

Hasil:



Gambar 1.23 Tampilan teks deskriptif dengan menggabungkan kolom `last_name` dan `salary` sebagai kolom “Pay”

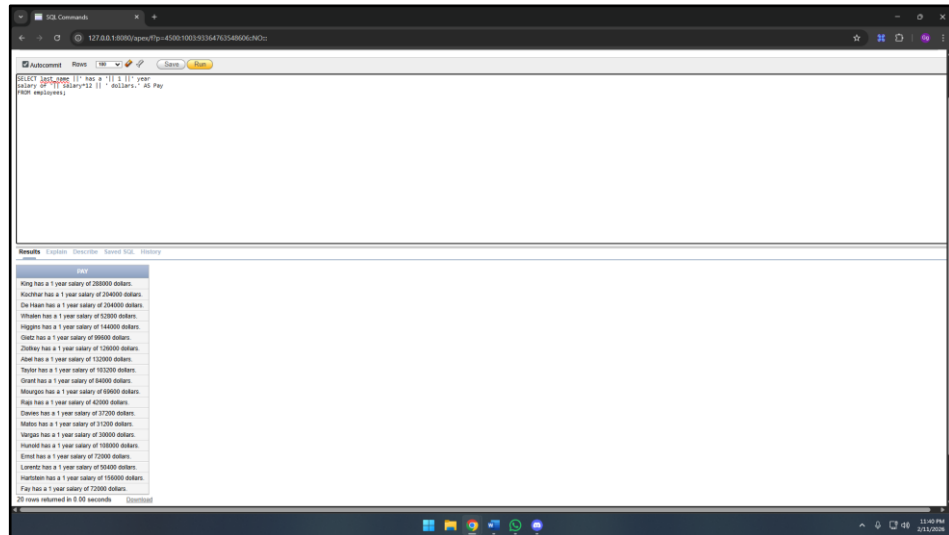
Penjelasan:

Query tersebut digunakan untuk menampilkan informasi gaji karyawan dengan menggabungkan nama karyawan dan nilai *salary* dalam satu kalimat. *Query* tersebut menggunakan operator `||` untuk menyusun kalimat keterangan gaji bulanan yang ditampilkan dalam satu kolom dengan alias “Pay”.

24. Query:

```
SELECT last_name || ' has a ' || 1 || ' year  
salary of ' || salary*12 || ' dollars.' AS Pay  
FROM employees;
```

Hasil:



Gambar 1.24 Tampilan teks deskriptif dengan menggabungkan kolom last_name dan salary*12 sebagai kolom “Pay

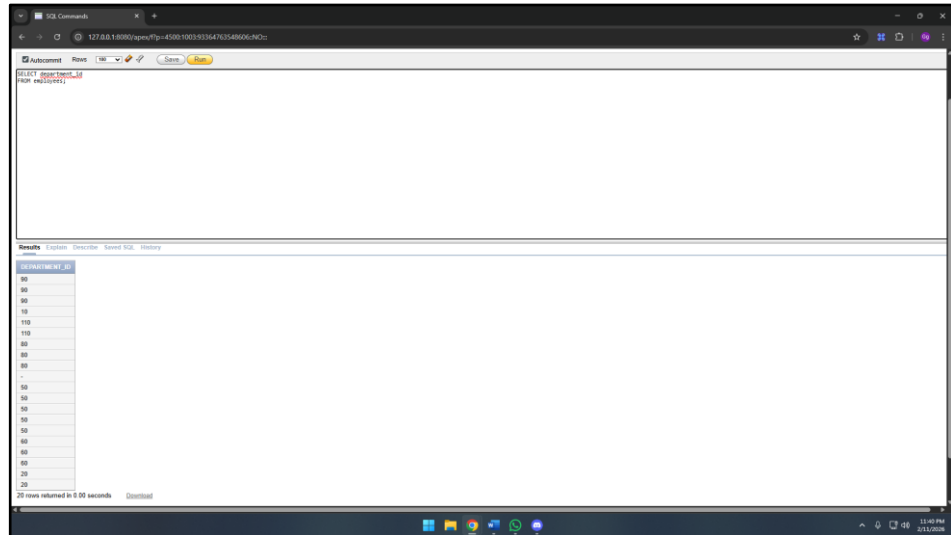
Penjelasan:

Query tersebut digunakan untuk menampilkan informasi gaji tahunan karyawan dengan menggabungkan nama karyawan dan hasil perhitungan gaji dalam satu tampilan. Query tersebut menggunakan operator || untuk Menyusun kalimat keterangan gaji tahunan yang ditampilkan dalam satu kolom dengan alias “Pay”.

25. Query:

```
SELECT department_id  
FROM employees;
```

Hasil:



Gambar 1.25 Tampilan kolom department_id

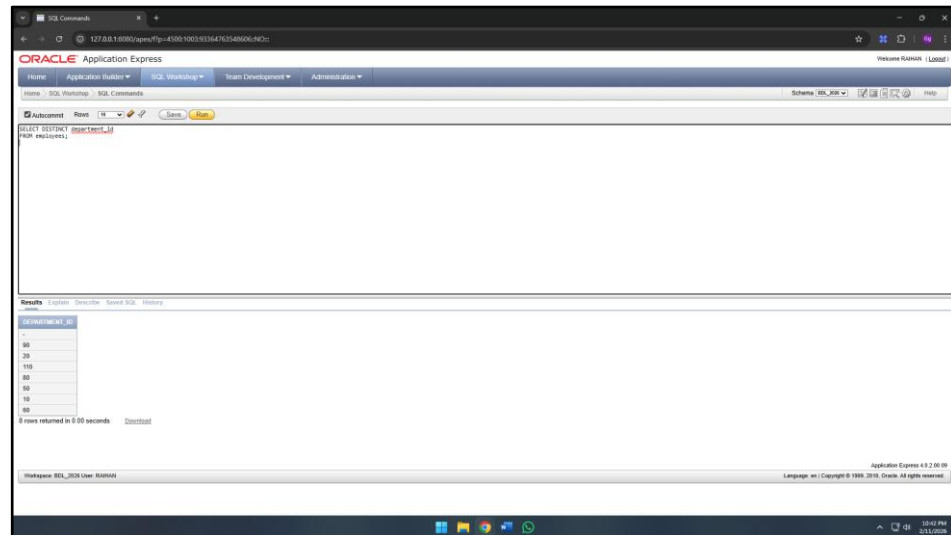
Penjelasan:

Query Query ini digunakan untuk menampilkan kolom department_id dari tabel employees. Query tersebut menampilkan daftar nilai department_id yang tersaji dalam satu kolom sebagai hasil eksekusi perintah.

26. Query:

```
SELECT DISTINCT department_id  
  
FROM employees;
```

Hasil:



Gambar 1.26 Tampilan daftar unik dari kolom `department_id` dari tabel `employees`

Penjelasan:

Query Query ini digunakan untuk menampilkan `department_id` yang unik dari tabel `employees`. Perintah `DISTINCT` memastikan bahwa setiap `department_id` hanya ditampilkan satu kali, sehingga pengguna dapat mengetahui daftar semua departemen yang memiliki karyawan tanpa pengulangan data.

27. Query:

```
SELECT employee_id, first_name, last_name  
FROM employees;
```

Hasil:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME
100	Steven	King
101	Neena	Kochhar
102	Lex	De Haan
103	Alexander	Russell
104	Bruce	Ernst
107	Diana	Lorentz
124	Kevin	Mearns
141	Tiana	Riss
142	Curtis	Dales
143	Randall	Munn
144	Peter	Vargas
149	Ellen	Zlotkey
174	Elan	Aish
176	Jonathan	Taylor
178	Kimberley	Grant
200	Jennifer	Whalen
201	Michael	Hartstein
202	Pat	Fay
205	Shelley	Higgins
206	William	Gietz

Gambar 1.27 Tampilan kolom `employee_id`, `first_name`, dan `last_name` dari tabel `employees`

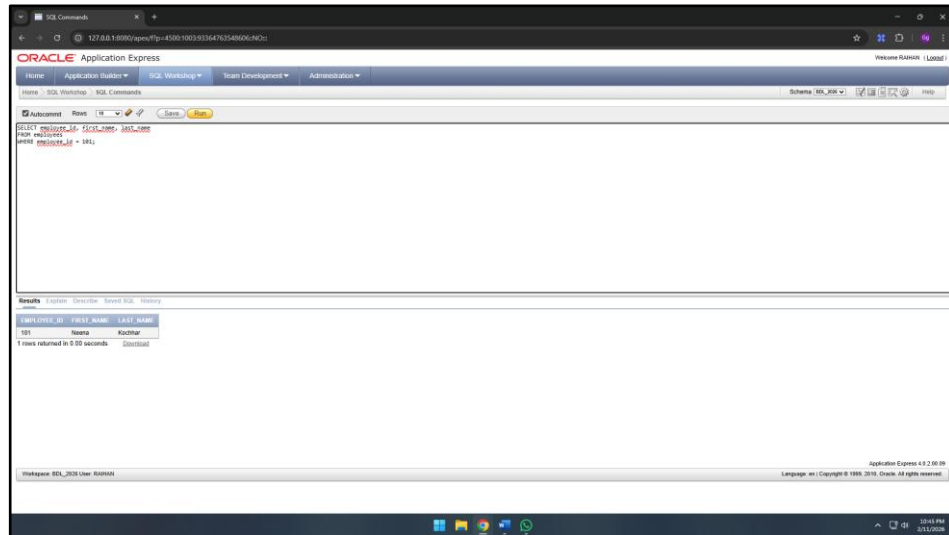
Penjelasan:

Query tersebut digunakan untuk menampilkan data karyawan yang terdiri dari `employee_id`, `first_name`, dan `last_name`. Perintah ini menyajikan identitas karyawan secara terstruktur dalam bentuk tabel, berdasarkan kolom-kolom yang telah dipilih. Dengan demikian, pengguna dapat melihat data dasar setiap karyawan secara rapi dan mudah dipahami.

28. Query:

```
SELECT employee_id, first_name, last_name  
FROM employees  
WHERE employee_id = 101;
```

Hasil:



Gambar 1.28 Tampilan kolom `employee_id`, `first_name`, dan `last_name` dengan `employee_id = 101`

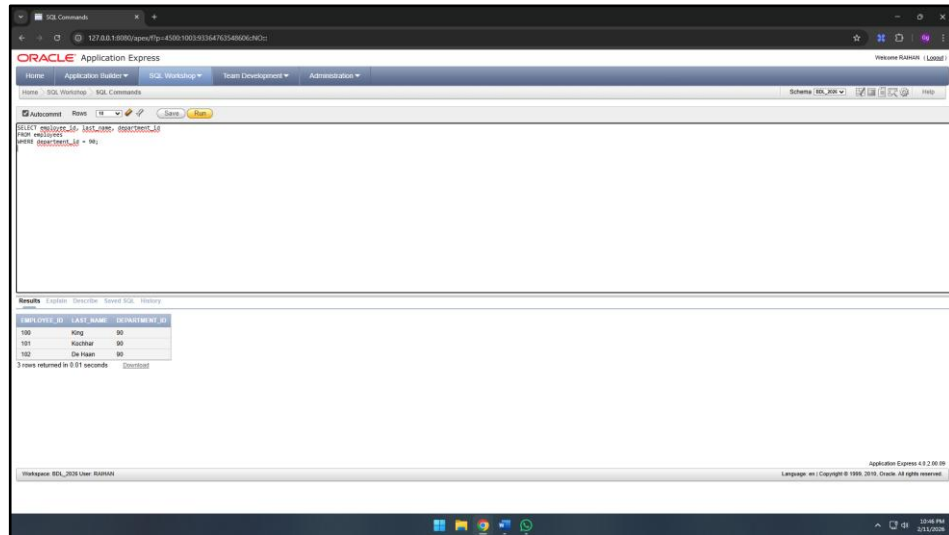
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `employee_id`, `first_name`, dan `last_name`. Perintah ini membatasi hasil sehingga hanya menampilkan data karyawan yang memenuhi kriteria yang ditetapkan, sehingga informasi yang diperoleh lebih spesifik dan relevan.

29. Query:

```
SELECT employee_id, last_name, department_id
FROM employees
WHERE department_id = 90;
```

Hasil:



The screenshot displays the Oracle Application Express interface. The SQL Command window contains the query: `SELECT employee_id, last_name, department_id FROM employees WHERE department_id = 90;`. The Results window shows the following data:

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
100	King	90
101	Kuchner	90
102	De Haan	90

3 rows returned in 0.21 seconds

Gambar 1.29 Tampilan kolom `employee_id`, `last_name`, dan `department_id` dengan `department_id = 90`

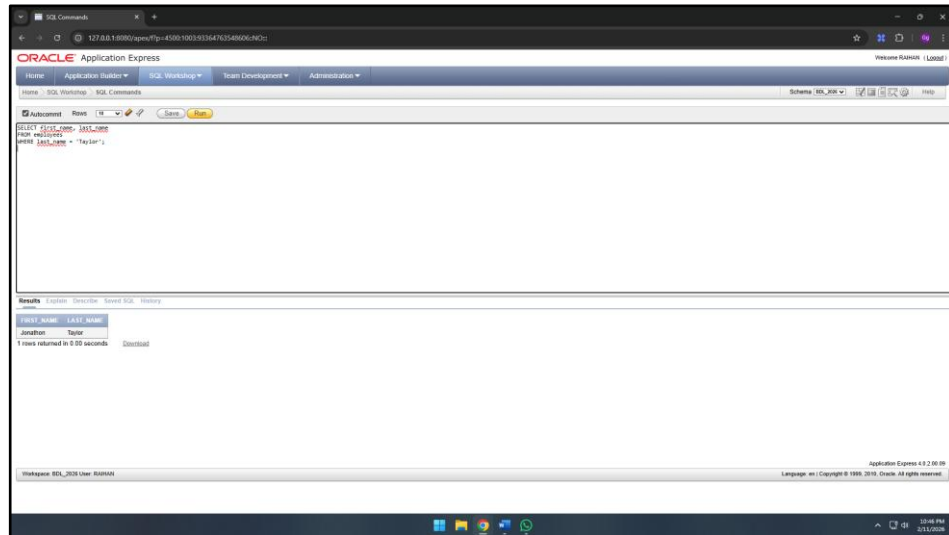
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `employee_id`, `last_name`, dan `department_id` dari tabel `employees`. Perintah ini membatasi hasil tampilan hanya pada karyawan dengan **`employee_id = 90`**, sehingga hanya baris data yang memenuhi kondisi tersebut yang akan ditampilkan. Dengan cara ini, pengguna dapat memperoleh informasi spesifik mengenai karyawan tertentu secara langsung.

30. Query:

```
SELECT first_name, last_name  
FROM employees  
WHERE last_name = 'Taylor';
```

Hasil:



Gambar 1.30 Tampilan kolom `first_name` dan `last_name` di mana `last_name` sama dengan `Taylor`

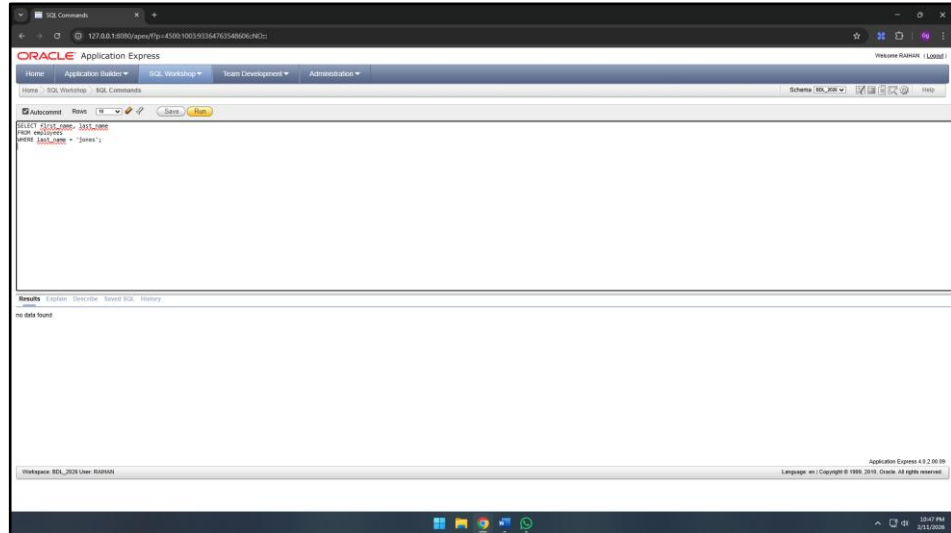
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `first_name` dan `last_name` dari tabel `employees` dengan kondisi `last_name` bernilai `Taylor`. Perintah ini membatasi hasil tampilan hanya pada karyawan yang memiliki nama belakang “Taylor”, sehingga hanya baris data yang memenuhi kriteria tersebut yang ditampilkan. Dengan demikian, pengguna dapat melihat informasi spesifik mengenai karyawan dengan nama belakang tertentu secara jelas.

31. Query:

```
SELECT first_name, last_name  
FROM employees  
WHERE last_name = 'jones';
```

Hasil:



Gambar 1.31 Tampilan kolom `first_name` dan `last_name` di mana `last_name` sama dengan `jones`

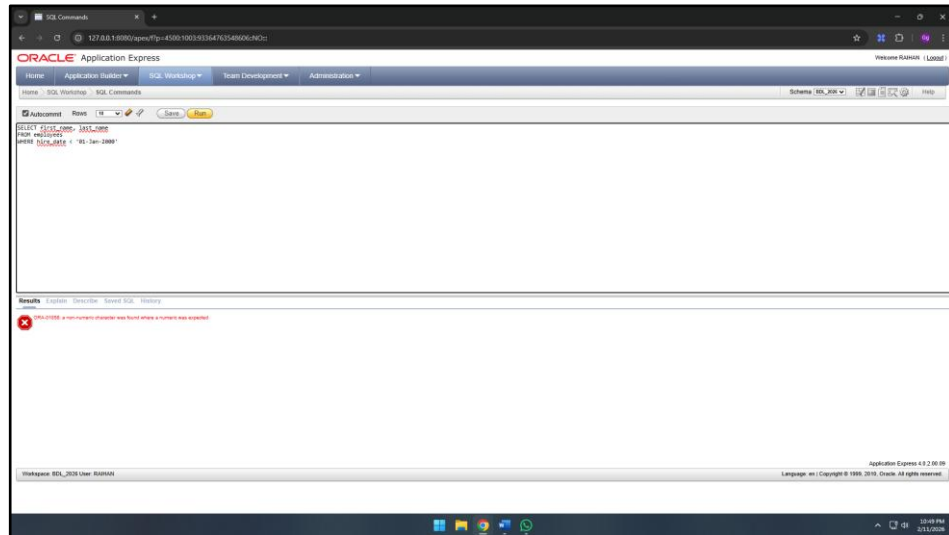
Penjelasan:

Query ini digunakan untuk menampilkan kolom `first_name` dan `last_name` dari tabel `employees` dengan kondisi `last_name` bernilai `jones`. Meskipun perintah SQL dijalankan dengan benar, hasil query tidak menampilkan data apa pun karena tidak ada karyawan dalam tabel yang memiliki nama belakang “Jones”. Akibatnya, eksekusi query menunjukkan *no data found*.

32. Query:

```
SELECT first_name, last_name  
FROM employees  
WHERE hire_date < '01-Jan-2000'
```

Hasil:



Gambar 1.32 Tampilan pesan *error* pada format 01-Jan-2000

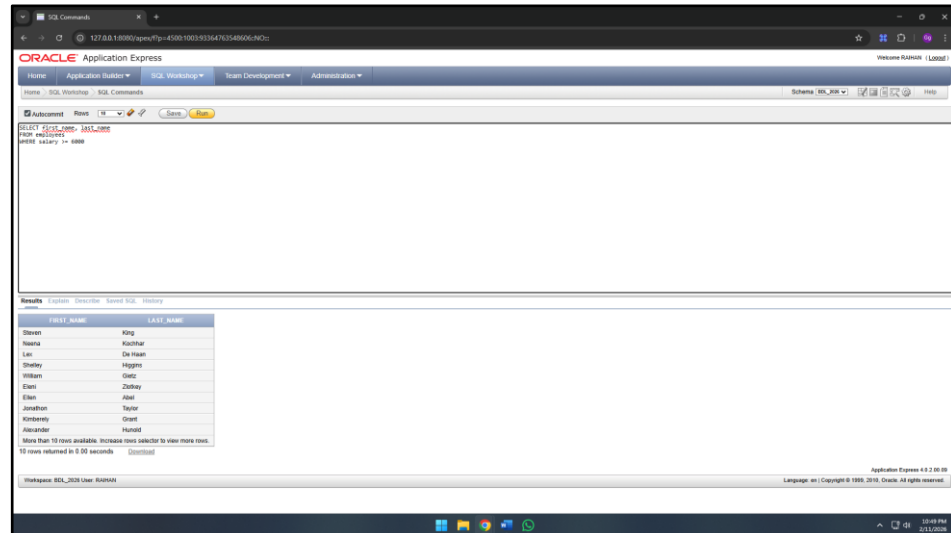
Penjelasan:

Query ini dimaksudkan untuk menampilkan kolom `first_name` dan `last_name` dari tabel `employees` dengan kondisi `hire_date` bernilai 01-Jan-2000. Namun, perintah SQL ini menghasilkan *error* karena format tanggal yang digunakan tidak sesuai dengan format tanggal yang dikenali. Akibatnya, *query* tidak dapat dijalankan dan tidak menampilkan hasil apa pun.

33. Query:

```
SELECT first_name, last_name  
FROM employees  
WHERE salary >= 6000
```

Hasil:



FIRST_NAME	LAST_NAME
Steven	King
Neena	Kochhar
Lex	De Haan
Shelley	Higgins
William	Giet
Ellen	Zlotney
Elan	Arlt
Jonathan	Taylor
Kimberley	Grant
Alexander	Russell

Gambar 1.33 Tampilan kolom `first_name` dan `last_name` di mana salary lebih dari atau sama dengan 6000

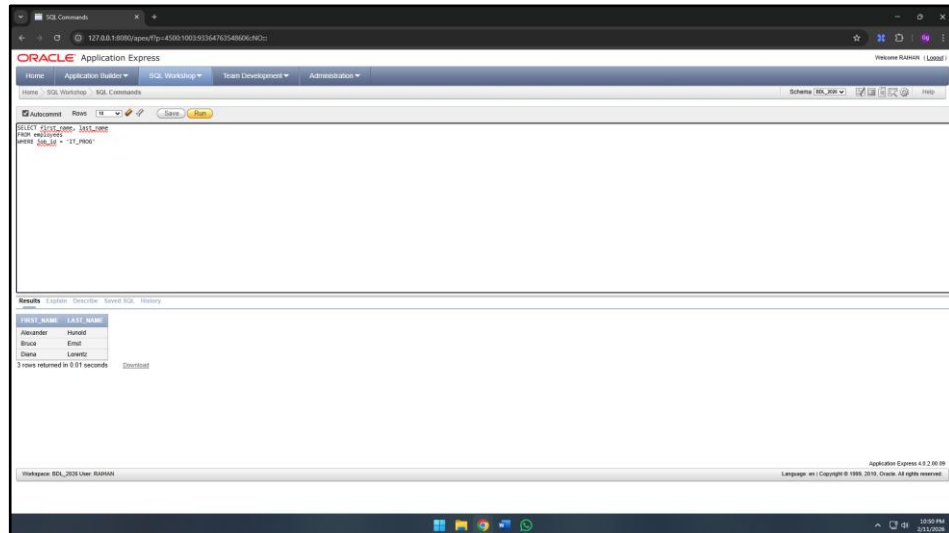
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `first_name` dan `last_name` dari tabel `employees` dengan kondisi `salary >= 6000`. Perintah ini menampilkan hanya karyawan yang memiliki gaji sama dengan atau lebih besar dari 6000. Dengan demikian, hasil *query* menampilkan daftar nama karyawan yang memenuhi kriteria gaji tersebut.

34. Query:

```
SELECT first_name, last_name  
FROM employees  
WHERE job_id = 'IT_PROG'
```

Hasil:



Gambar 1.34 Tampilan kolom `first_name` dan `last_name` di mana `job_id` sama dengan `IT_PROG`

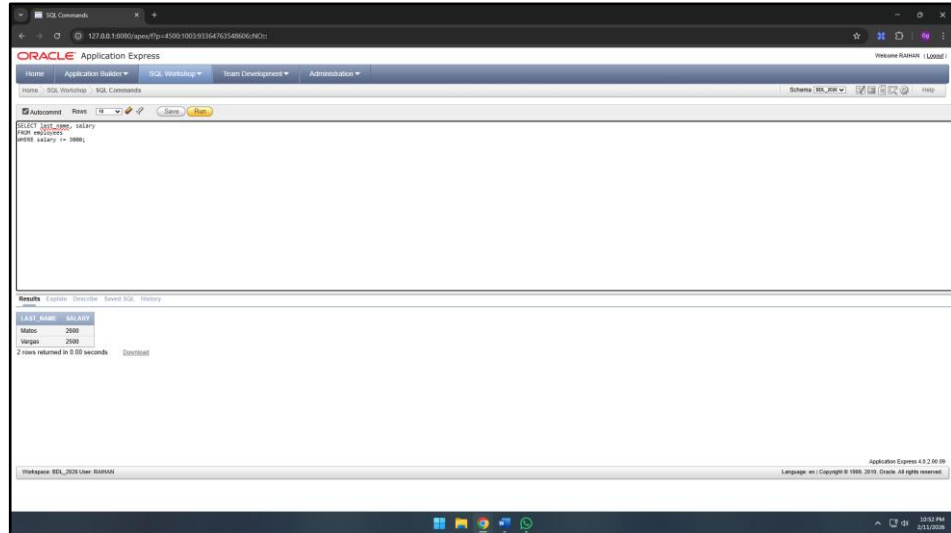
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `first_name` dan `last_name` dari tabel `employees` dengan kondisi `job_id = 'IT_PROG'`. Perintah ini membatasi hasil sehingga hanya menampilkan karyawan yang memiliki jabatan sebagai “IT Programmer”.

35. Query:

```
SELECT last_name, salary
FROM employees
WHERE salary <= 3000;
```

Hasil:



Gambar 1.35 Tampilan kolom `last_name` dan `salary` di mana `salary` kurang dari atau sama dengan 3000

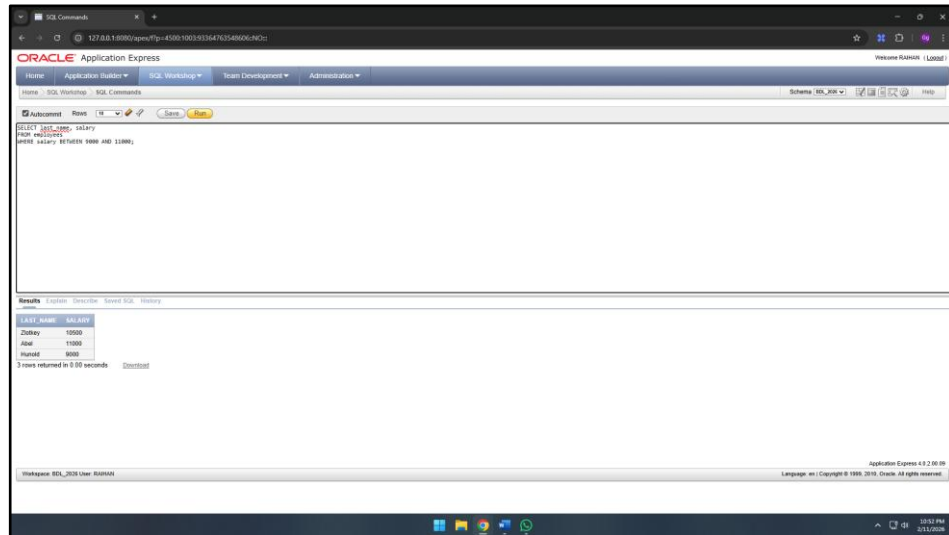
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `last_name` dan `salary` dari tabel `employees` dengan kondisi `salary <= 3000`. Perintah ini menampilkan hanya karyawan yang memiliki gaji kurang dari atau sama dengan 3000. Dengan begitu, hasil *query* menunjukkan daftar nama karyawan yang memenuhi kriteria gaji tersebut secara jelas.

36. Query:

```
SELECT last_name, salary
FROM employees
WHERE salary BETWEEN 9000 AND 11000;
```

Hasil:



The screenshot shows the Oracle Application Express interface. The query editor at the top contains the SQL query: `SELECT last_name, salary FROM employees WHERE salary BETWEEN 9000 AND 11000;`. The 'Results' section below shows a table with 3 rows returned in 0.00 seconds. The table has two columns: last_name and salary.

last_name	salary
Zlotkey	10000
Abel	11000
Hunee	9000

Gambar 1.36 Tampilan kolom `last_name` dan `salary` di mana nilai `salary` berada di antara 9000 dan 11000

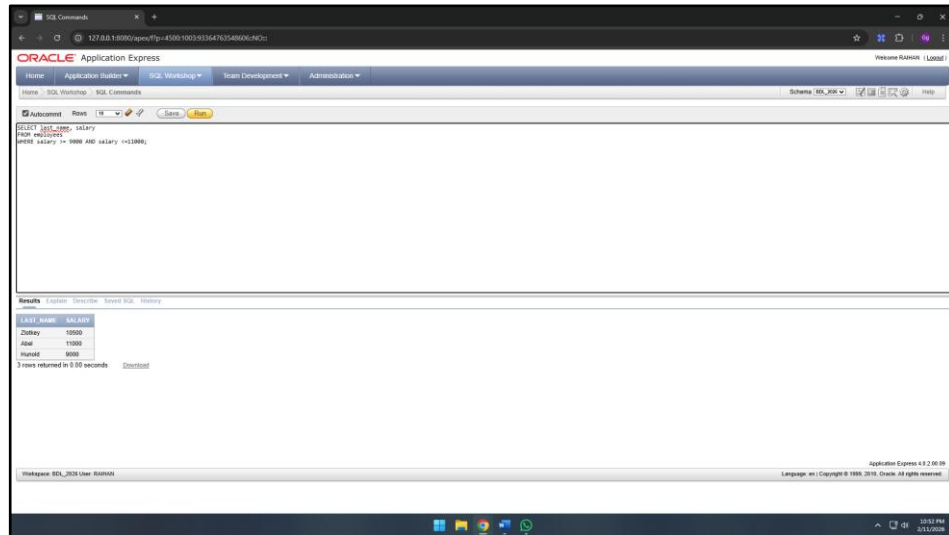
Penjelasan:

Query ini digunakan untuk menampilkan kolom `last_name` dan `salary` dari tabel `employees` dengan kondisi `salary BETWEEN 9000 AND 11000`. perintah ini menampilkan hanya karyawan yang memiliki gaji berada di dalam rentang tersebut. Dengan demikian, hasil *query* menunjukkan daftar nama karyawan beserta gaji mereka sesuai kriteria yang telah ditentukan.

37. Query:

```
SELECT last_name, salary
FROM employees
WHERE salary >= 9000 AND salary <=11000;
```

Hasil:



The screenshot shows the Oracle Application Express interface. The query editor at the top contains the SQL query: `SELECT last_name, salary FROM employees WHERE salary >= 9000 AND salary <=11000;`. The results pane below shows a table with two columns: last_name and salary. The table contains three rows of data: Zlotkey with salary 10500, Abad with salary 11000, and Harvath with salary 9000. The status bar at the bottom indicates that 3 rows were returned in 0.30 seconds.

last_name	salary
Zlotkey	10500
Abad	11000
Harvath	9000

Gambar 1.37 Tampilan kolom last_name dan salary di mana dengan salary >= 9000 AND salary <=11000

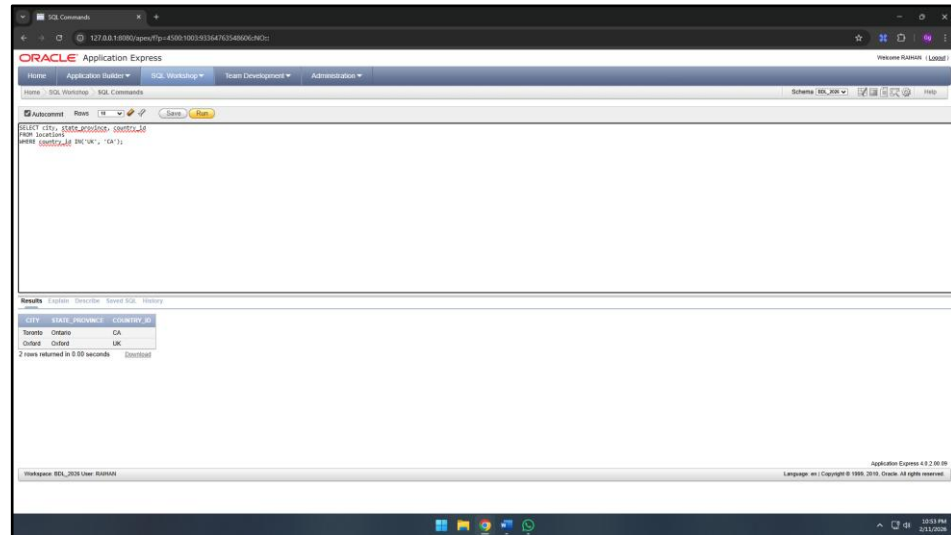
Penjelasan:

Query ini digunakan untuk menampilkan kolom last_name dan salary dari tabel employees dengan kondisi salary >= 9000 AND salary <=11000. Perintah ini hanya menampilkan karyawan yang memiliki gaji di dalam rentang nilai tersebut. Dengan demikian, hasil query menyajikan daftar nama karyawan beserta gaji mereka sesuai kriteria yang telah ditetapkan.

38. Query:

```
SELECT city, state_province, country_id
FROM locations
WHERE country_id IN('UK', 'CA');
```

Hasil:



The screenshot shows the Oracle Application Express interface. The SQL command window contains the query: `SELECT city, state_province, country_id FROM locations WHERE country_id IN('UK', 'CA');`. The Results window displays the following data:

CITY	STATE_PROVINCE	COUNTRY_ID
Toronto	Ontario	CA
Oxford	Oxford	UK

2 rows returned in 0.00 seconds

Gambar 1.38 Tampilan kolom `city`, `state_province`, dan `country_id` dengan nilai kolom `country_id` adalah UK atau CA

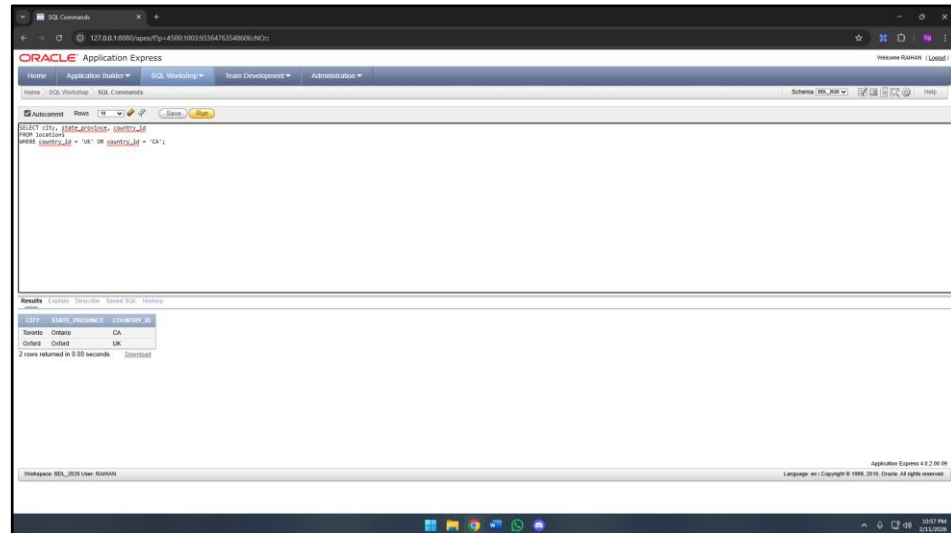
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `city`, `state_province`, dan `country_id` dari tabel `locations` dengan kondisi `country_id` bernilai UK atau CA. Hasil *query* menampilkan data lokasi yang berada di negara Inggris dan Kanada sesuai dengan kriteria yang ditentukan.

39. Query:

```
SELECT city, state_province, country_id
FROM locations
WHERE country_id = 'UK' OR country_id = 'CA';
```

Hasil:



CITY	STATE_PROVINCE	COUNTRY_ID
Toronto	Ontario	CA
Oxford	Oxford	UK

2 rows returned in 0.00 seconds

Gambar 1.39 Tampilan kolom city, state_province, dan country_id di mana nilai kolom country_id adalah UK atau CA

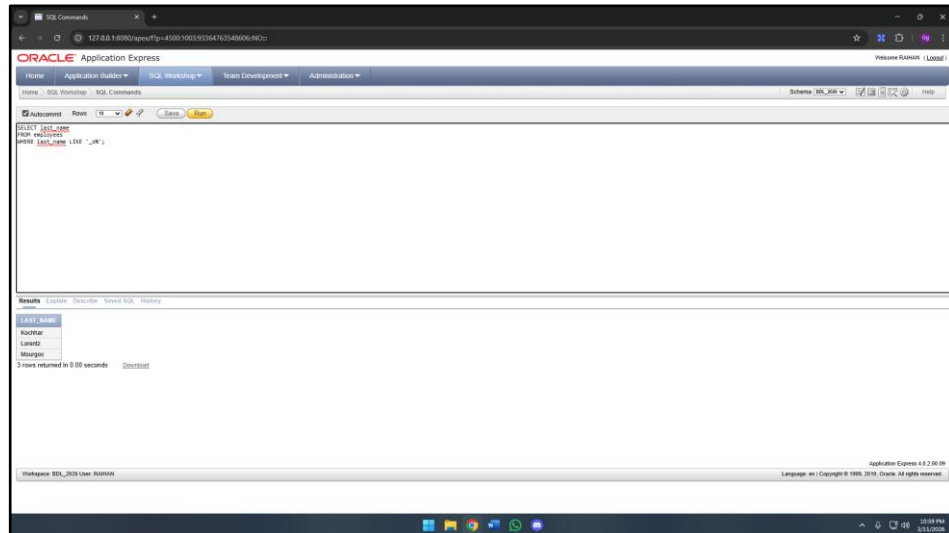
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom city, state_province, dan country_id dari tabel locations dengan kondisi country_id bernilai UK atau CA. Hasil query menampilkan data lokasi yang berada di negara United Kingdom dan Canada sesuai kriteria yang ditentukan. Dengan demikian, hasil query memberikan daftar kota dan provinsi yang terkait dengan kedua negara tersebut.

40. Query:

```
SELECT last_name  
FROM employees  
WHERE last_name LIKE '_o%';
```

Hasil:



Gambar 1.40 Tampilan kolom `last_name` dengan nilai kolom `last_name` mengandung karakter huruf `o` setelah huruf pertama

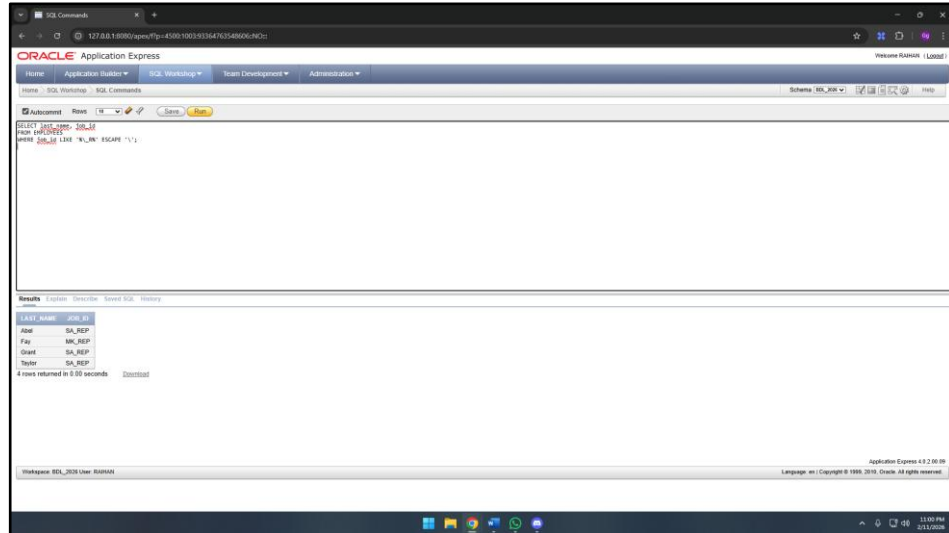
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `last_name` dari tabel `employees` dengan kondisi nilai memiliki satu karakter huruf bebas di posisi pertama, diikuti oleh huruf `o` pada posisi kedua dan karakter huruf apa pun setelahnya. Hasil query menampilkan data nama belakang karyawan yang sesuai dengan kriteria yang ditentukan.

41. Query:

```
SELECT last_name, job_id
FROM EMPLOYEES
WHERE job_id LIKE '%\_R%' ESCAPE '\';
```

Hasil:



Gambar 1.41 Tampilan kolom `last_name`, dan `job_id` yang mengandung karakter `_R` pada kolom `job_id`

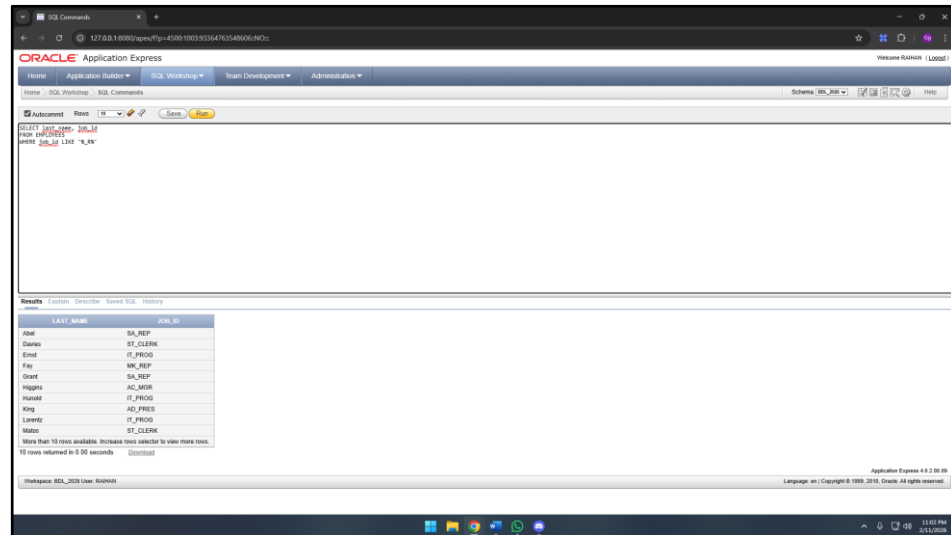
Penjelasan:

Query ini digunakan untuk menampilkan kolom `last_name` dan `job_id` dari tabel `employees` dengan kondisi `job_id` mengandung karakter huruf apa pun sebelum tanda *underscore* “`_`” yang diikuti oleh huruf `R`, lalu diikuti oleh karakter huruf apa pun setelahnya. Penggunaan klausa `ESCAPE '\'` bertujuan agar tanda *underscore* “`_`” dianggap sebagai karakter biasa dan bukan sebagai simbol khusus dalam pola pencarian. Dengan demikian, hasil *query* memberikan daftar karyawan yang memenuhi kriteria huruf pada nama belakangnya.

42. Query:

```
SELECT last_name, job_id
FROM EMPLOYEES
WHERE job_id LIKE '%_R%'
```

Hasil:



The screenshot shows the Oracle Application Express interface. The SQL command window contains the query: `SELECT last_name, job_id FROM EMPLOYEES WHERE job_id LIKE '%_R%'`. The results pane displays a table with two columns: **LAST_NAME** and **JOB_ID**. The data is as follows:

LAST_NAME	JOB_ID
Abel	SA_REP
Davies	ST_CLERK
Ernst	IT_PROG
Fay	HR_REP
Grant	SA_REP
Higgins	AC_MGR
Hunee	IT_PROG
King	AD_PRES
Lorentz	IT_PROG
Martin	ST_CLERK

Below the table, it states: "More than 10 rows available. Increase row selector to view more rows. 10 rows returned in 0.00 seconds".

Gambar 1.42 Tampilan kolom `last_name`, dan `job_id` yang memiliki satu karakter diikuti karakter huruf R pada kolom `job_id`

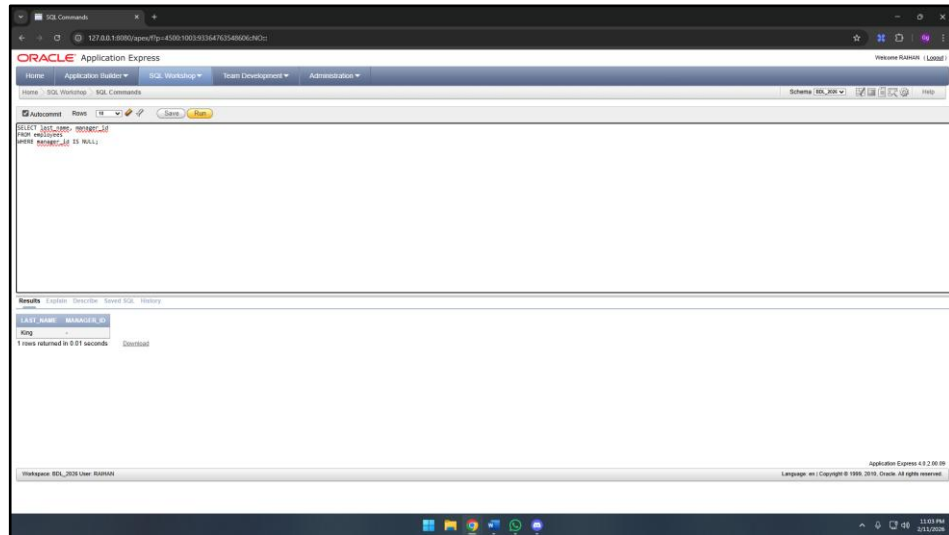
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom `last_name` dan `job_id` dari tabel `employees` dengan kondisi `job_id` mengandung huruf R pada posisi tertentu. Perintah ini menampilkan hanya karyawan yang memiliki `job_id` sesuai pola tersebut. Dengan demikian, hasil *query* memberikan daftar nama karyawan beserta jabatan mereka yang memenuhi kriteria huruf pada `job_id`.

43. Query:

```
SELECT last_name, manager_id
FROM employees
WHERE manager_id IS NULL;
```

Hasil:



Gambar 1.43 Tampilan last_name dan manager_id pada tabel employees yang memiliki nilai manager_id NULL

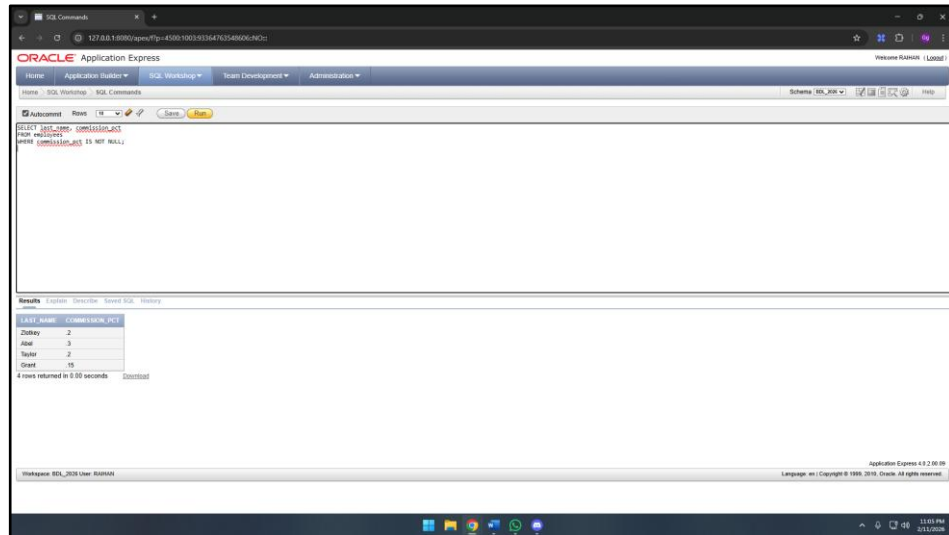
Penjelasan:

Query tersebut digunakan untuk menampilkan kolom last_name dan manager_id dari tabel employees dengan kondisi manager_id IS NULL. Hasil query menampilkan data karyawan yang memiliki kolom manager_id kosong sesuai dengan kondisi pada query.

44. Query:

```
SELECT last_name, commission_pct  
FROM employees  
WHERE commission_pct IS NOT NULL;
```

Hasil:



last_name	commission_pct
Deity	2
Abal	3
Taylor	2
Grant	15

Gambar 1.44 Tampilan last_name dan commission_pct pada tabel employees yang memiliki nilai commission_pct tidak NULL

Penjelasan:

Query tersebut digunakan untuk menampilkan kolom last_name dan commission_pct dari tabel employees dengan kondisi commission_pct IS NOT NULL. Hasil query menampilkan data karyawan yang memiliki kolom commission_pct terisi sesuai dengan kondisi pada query.

1.4 Latihan

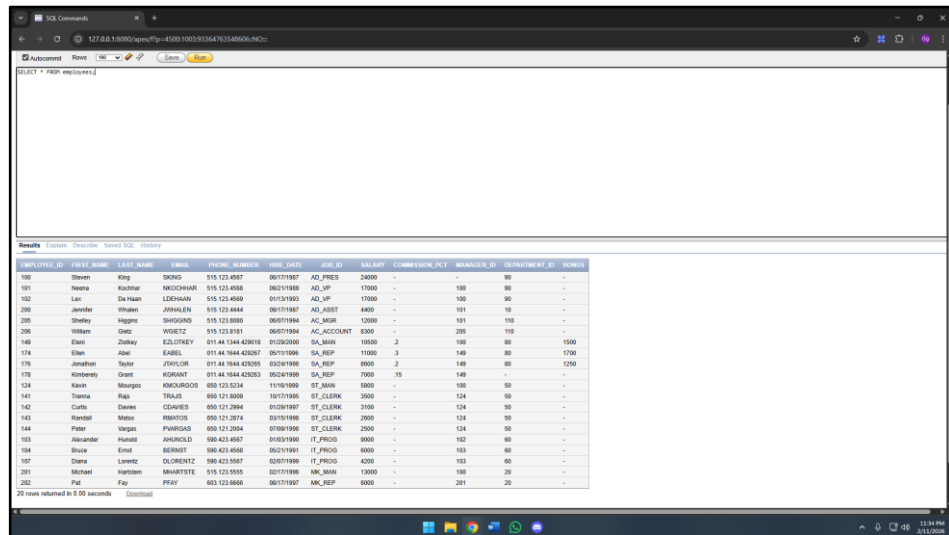
Adapun latihan pada bab ini adalah sebagai berikut:

1. Menampilkan seluruh data dari tabel `employees`.

Query:

```
SELECT * FROM employees;
```

Hasil:



EMPLOYEE_ID	LAST_NAME	FIRST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	BONUS
100	Deena	King	SKING	515.123.4567	06/17/1987	AD_PRECS	24000	-	-	-	90	-
101	Naima	Kochhar	NKOCHHAR	515.123.4568	06/17/1989	AD_VP	17000	-	-	100	90	-
102	Lai	De Haan	LDEHAAN	515.123.4569	01/13/1993	AD_VP	17000	-	-	100	90	-
200	Jennifer	Winters	JWINTERS	515.123.4444	06/17/1987	AD_ASST	4400	-	-	101	10	-
201	Shelley	Higgins	SHIGGINS	515.123.8080	06/07/1994	AC_MGR	12000	-	-	101	110	-
206	William	Gietz	WGIEZT	515.123.8181	06/07/1994	AC_ACCOUNT	8300	-	-	205	110	-
140	Elvis	Storley	ELSTORLEY	011.44.1644.430218	01/02/2004	SA_MGR	15000	2	-	130	80	1500
174	Elvis	Abel	EABEL	011.44.1644.430237	05/17/1996	SA_REP	11000	3	-	140	80	1700
176	Jonathan	Taylor	JTAYLOR	011.44.1644.430295	03/24/1998	SA_REP	8000	2	-	140	80	1200
178	Kenneth	Grant	KGRANT	011.44.1644.430293	05/21/1996	SA_REP	7000	15	-	140	-	-
124	Karen	Maunga	KMAUNGA	650.123.5234	11/08/1999	ST_MGR	5800	-	-	100	50	-
141	Tanna	Rais	TRAIS	650.121.8009	10/17/1995	ST_CLERK	3600	-	-	124	50	-
142	Carlin	Daems	CDAEMS	650.121.2044	01/09/1997	ST_CLERK	3100	-	-	124	50	-
143	Randall	Mateo	RMATEO	650.121.2074	03/10/1998	ST_CLERK	2900	-	-	124	50	-
144	Peter	Vargas	PVARGAS	650.121.2064	07/08/1998	ST_CLERK	2500	-	-	124	50	-
103	Alexander	Hunold	AHUNOLD	590.423.4567	01/03/1996	IT_PROG	6000	-	-	102	60	-
104	Bruce	Ernst	BERNST	590.423.4568	06/21/1997	IT_PROG	6000	-	-	103	60	-
107	Diana	Lorentz	DLLORENTZ	590.423.5067	02/07/1999	IT_PROG	4200	-	-	103	60	-
201	Michael	Hartstein	MHARTSTE	515.123.5555	02/17/1996	MK_MGR	12000	-	-	100	20	-
202	Pai	Fay	PFAY	650.123.6666	06/17/1997	MK_REP	6000	-	-	201	20	-

Gambar 1.45 Tampilan seluruh data dari tabel `employees`

Penjelasan:

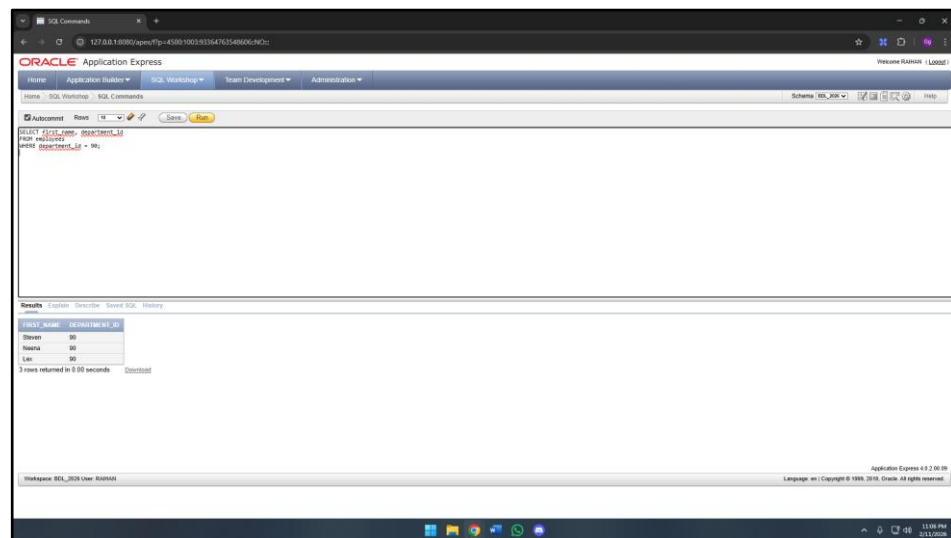
Query ini digunakan untuk menampilkan seluruh kolom beserta data yang terdapat pada tabel `employees`. Perintah `SELECT` berfungsi untuk mengambil data dari tabel, sedangkan simbol “*” menandakan bahwa semua kolom akan ditampilkan. Bagian `FROM employees` menunjukkan bahwa sumber data yang diambil berasal dari tabel `employees`.

- Menampilkan hanya kolom `first_name` dan `department_id` dari tabel `employees` untuk karyawan yang memiliki `department_id` 90.

Query:

```
SELECT first_name, department_id
FROM employees
WHERE department_id = 90;
```

Hasil:



Gambar 1.46 Tampilan kolom `first_name` dan `department_id` untuk karyawan yang memiliki `department_id` 90

Penjelasan:

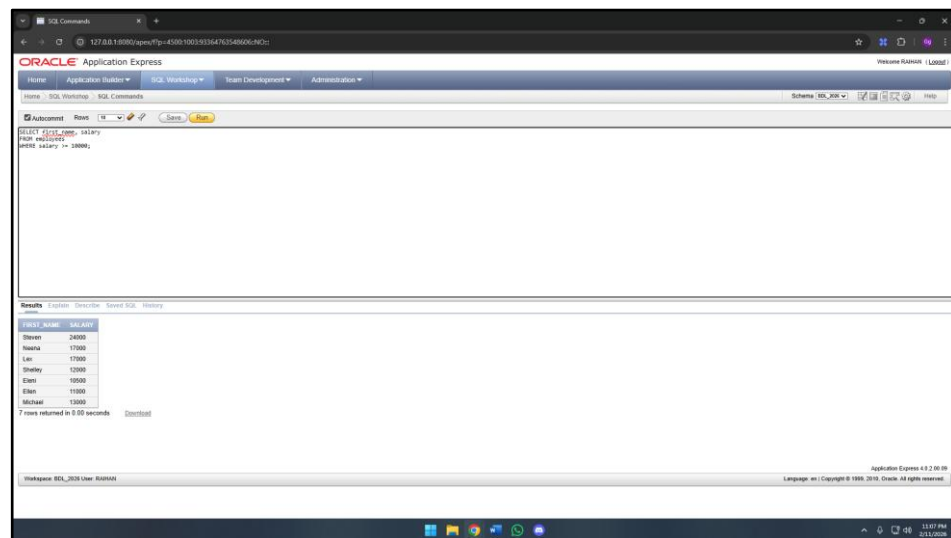
Query ini berfungsi untuk menampilkan kolom `first_name` dan `department_id` dari tabel `employees` dengan syarat `department_id = 90`. Hasil *query* menampilkan data karyawan yang memiliki `department_id = 90` sesuai dengan kondisi yang ditentukan. Dengan begitu, hasil *query* memperlihatkan daftar nama depan karyawan yang termasuk dalam departemen tertentu sesuai kriteria yang ditetapkan..

3. Menampilkan hanya kolom `first_name` dan `salary` karyawan yang memiliki `salary` lebih dari atau sama dengan 10000.

Query:

```
SELECT first_name, salary
FROM employees
WHERE salary >= 10000;
```

Hasil:



first_name	salary
Steven	24000
Neena	17000
Lex	17000
Shelley	12000
Elina	10500
Ehan	11000
Michael	12000

Gambar 1.47 Tampilan kolom `first_name` dan `salary` karyawan yang memiliki `salary` $\geq$ 10000

Penjelasan:

Query ini menampilkan kolom `first_name` dan `salary` dari tabel `employees` dengan kondisi `salary` $\geq$ 10000. perintah ini menampilkan hanya karyawan yang memiliki gaji sama dengan atau lebih dari 10000. Dengan demikian, hasil *query* memberikan daftar nama karyawan beserta gaji mereka yang memenuhi batas minimal yang ditentukan..

1.5 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa SQL (Structured Query Language) merupakan bahasa yang memiliki peran penting dalam pengelolaan basis data relasional. SQL memungkinkan pengguna untuk berinteraksi dengan basis data secara terstruktur melalui berbagai perintah yang disediakan. Dengan adanya SQL, proses pengambilan dan pengolahan data dapat dilakukan dengan lebih sistematis, sehingga memudahkan pengguna dalam mengelola informasi yang tersimpan di dalam basis data.

Salah satu perintah dasar yang sering digunakan dalam SQL adalah SELECT. Perintah ini berfungsi untuk menentukan data atau kolom apa saja yang ingin ditampilkan dari suatu tabel. Penggunaan SELECT membantu pengguna dalam menampilkan data yang dibutuhkan tanpa harus menampilkan seluruh isi tabel, sehingga informasi yang diperoleh menjadi lebih fokus dan sesuai dengan tujuan.

Selain SELECT, klausa WHERE juga memiliki peran yang sangat penting dalam pengolahan data. WHERE digunakan untuk memberikan batasan atau kondisi tertentu terhadap data yang akan ditampilkan. Dengan mengombinasikan SELECT dan WHERE, pengguna dapat memperoleh data yang lebih spesifik dan relevan, sehingga mendukung proses analisis data secara lebih terarah dan efektif.

DAFTAR PUSTAKA

- Catayoc, A. (2025). *Comparative analysis of table aliasing in SQL queries: Functional and semantic implications*. ResearchGate.
- Ciptayani, R. (2020). *Konsep DISTINCT dan NULL dalam Structured Query Language (SQL)*.
- Fitri, A. (2020). *Penggunaan operator LIKE dalam pencarian data pada SQL*.
- Oracle. (2020). *SQL language reference*. Oracle Database 21c Documentation.
- Prayoga, A., et al. (2023). *Implementasi operator BETWEEN dalam penyaringan data SQL*.
- Santoso, B. (2021). *Dasar-dasar Structured Query Language (SQL)*.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill Education.